



**UNIVERSITÀ
DEGLI STUDI
DI UDINE**
HIC SUNT FUTURA

DIPARTIMENTO DI SCIENZE MATEMATICHE, INFORMATICHE E FISICHE

TESI DI LAUREA IN
INFORMATICA

RAM-USB

Progettazione e implementazione di un server di backup multi utente Zero-Knowledge e Zero-Trust

CANDIDATO

Francesco Verrengia

RELATORE

Prof. Ivan Scagnetto

Anno accademico 2025-2026

CONTATTI DELL'ISTITUTO

Dipartimento di Scienze Matematiche, Informatiche e Fisiche

Università degli Studi di Udine

Via delle Scienze, 206

33100 Udine — Italia

+39 0432 558400

<https://www.dmif.uniud.it/>

Sommario

RAM-USB è un servizio di backup multi-utente, geo-distribuito e accessibile da remoto, progettato secondo tre principi di sicurezza: zero-knowledge, nessun componente lato server accede mai al contenuto dei backup in chiaro; zero-trust, nessun componente si fida implicitamente dei dati ricevuti da un altro; defense-in-depth, ogni strato rivalida indipendentemente l'input ricevuto. Il progetto nasce come caso di studio approfondito sulla progettazione di sistemi di backup distribuiti.

Il sistema è un'architettura a microservizi n-tier: ogni registrazione o login attraversa quattro processi distinti, ciascuno dei quali rivalida l'input senza fidarsi dello strato precedente, mentre l'accesso allo spazio di backup, isolato per utente tramite chroot dinamico, resta possibile solo dopo l'autenticazione, su una rete privata mesh che applica confini di raggiungibilità distinti per ogni fase del ciclo di vita dell'utente. I requisiti sono raccolti in un unico documento tracciabile fino al codice, con un modello a spirale per l'analisi e un modello a V per l'implementazione.

L'analisi critica delle proprietà di sicurezza quantifica il costo di un attacco offline alle password, giorni per la lunghezza minima ammessa, ordini di grandezza superiori per password più lunghe, e verifica componente per componente che la compromissione isolata di un servizio non si propaghi agli altri. Ogni scelta architetturale è messa a bilancio con il prezzo che comporta, dalla latenza introdotta dalla separazione in più processi alla complessità della rivalidazione indipendente.

Il sistema è stato distribuito e verificato end-to-end su un'infrastruttura reale. I suoi limiti del sistema sono dichiarati insieme agli sviluppi che potrebbero mitigarli.

Indice

1	Introduzione	1
1.1	Contesto e motivazione	1
1.2	Problema affrontato	1
1.3	Obiettivi e perimetro	2
1.4	Metodologia	2
1.5	Struttura della tesi	2
2	Stato dell'arte e contesto tecnologico	5
2.1	Principi di sicurezza di riferimento	5
2.2	Soluzioni di backup esistenti	5
2.3	Paralleli in altri domini a conoscenza zero	6
2.4	Reti mesh e zero-trust networking	6
2.5	Posizionamento di RAM-USB	6
3	Requisiti: metodologia e casi d'uso	9
3.1	Metodologia dei requisiti	9
3.2	Tracciabilità dei requisiti	9
3.3	Requisiti utente e casi d'uso	10
4	Architettura generale e trust zones	13
4.1	Overview architetturale	13
4.1.1	Entry-Hub	14
4.1.2	Security-Switch	14
4.1.3	Database-Vault	14
4.1.4	Storage-Service	15
4.1.5	Network-Manager	16
4.1.6	Certificate-Authority	16
4.1.7	Headscale	16
4.1.8	Mosquitto	17
4.1.9	Metrics-Collector	17
4.1.10	Grafana	17
4.2	Modello di comunicazione inter-servizio	17
4.3	Trust zones e confini di sicurezza	18
4.4	PKI e gestione dei certificati	18
5	Autenticazione e gestione delle identità	21
5.1	Ciclo di vita della richiesta	21
5.2	Cifratura e hashing delle credenziali	22
5.2.1	Robustezza della password	24
5.3	Registrazione	26
5.4	Login	27
5.5	Orchestrazione post-autenticazione	29

6	Storage e backup dei file	31
6.1	Modello di accesso SFTP-only	31
6.2	Isolamento e chroot dinamico	32
6.3	Backup e restore	32
7	Rete e coordinamento della mesh	35
7.1	Politiche di rete e confini di reachability	35
7.2	Ciclo di vita dei grant di accesso	35
7.3	Il paradosso del coordinatore: Headscale come servizio a sé	37
7.4	CG-NAT e la necessità di un endpoint pubblico dedicato	38
8	Osservabilità e metriche	41
8.1	Pubblicazione delle metriche	41
8.2	Ricezione e controllo di coerenza	42
8.3	Persistenza delle serie temporali	42
8.4	Visualizzazione delle metriche	42
9	Testing e validazione	45
9.1	Il modello V e i quattro livelli di test	45
9.2	Strategia di integrazione bottom-up	45
9.3	TDD: criteri di ingresso e uscita	46
9.4	Copertura e lacune del piano di test	46
9.5	Verifica automatica pre-commit	47
9.6	Risultati: verifiche completate	47
10	Deployment	49
10.1	Containerizzazione e supervisione s6-overlay	49
10.2	Perché non tutti i container sono distroless	49
10.3	Docker locale vs Proxmox VE: due livelli di isolamento	50
10.4	Topologia distribuita	51
10.5	Alta disponibilità e resilienza: stato attuale e limiti	51
11	Analisi critica delle proprietà di sicurezza	53
11.1	Scenari di attacco e mitigazioni	53
11.1.1	Enumerazione degli indirizzi email in registrazione	54
11.1.2	Accesso amministrativo alle credenziali	54
11.1.3	Verifica offline di un indirizzo a partire da una copia del database	55
11.2	Costo di un attacco alle password	55
11.3	Isolamento dei fallimenti	56
11.4	Costo delle scelte architetturali	57
12	Conclusioni	59
12.1	Risultati raggiunti	59
12.2	Limiti noti	59
12.3	Sviluppi futuri	60
12.3.1	Mitigazione dell'enumerazione degli indirizzi email	60
12.3.2	Le due regole della policy delle password	60
12.3.3	Hash con chiave per la ricerca dell'email	61
12.3.4	Altri requisiti "dovrebbe" non implementati	61

A	Elenco completo dei requisiti	63
A.1	Requisiti di sistema funzionali	63
A.1.1	Client	63
A.1.2	Entry-Hub	64
A.1.3	Security-Switch	65
A.1.4	Database-Vault	66
A.1.5	Storage-Service	67
A.1.6	Network-Manager	68
A.1.7	Certificate-Authority	70
A.1.8	Sistema di monitoraggio	70
A.1.9	Infrastruttura di rete e PKI	70
A.2	Requisiti non funzionali	71
A.2.1	Di prodotto	71
A.2.2	Organizzativi	71
A.2.3	Esterni	72
A.3	Requisiti di dominio	72

1

Introduzione

1.1 Contesto e motivazione

RAM-USB è un servizio di backup multi-utente, geo-distribuito e accessibile da remoto, progettato secondo tre principi di sicurezza: zero-knowledge, zero-trust e defense-in-depth [1]. Zero-knowledge significa che nessun componente lato server può accedere in chiaro al contenuto dei backup; zero-trust, che nessun componente si fida implicitamente dei dati ricevuti da un altro, anche se già autenticato; defense-in-depth, che ogni strato rivalida indipendentemente l'input, senza dare per scontato che lo strato precedente lo abbia già fatto.

L'obiettivo del progetto non è competere con soluzioni commerciali di backup, ma costituire un caso di studio approfondito sulla progettazione sicura di sistemi di backup distribuiti. La correttezza e la trasparenza della progettazione contano quindi più della velocità di consegna o della copertura delle funzionalità: una scelta che attraversa l'intera tesi, dalla metodologia dei requisiti nel capitolo 3 fino al bilancio critico delle proprietà di sicurezza nel Capitolo 11.

1.2 Problema affrontato

Due scelte di design, deliberate fin dalla SRS [1], derivano direttamente da questo obiettivo.

La prima riguarda l'autenticazione. Un protocollo zero-knowledge come SRP eviterebbe di trasmettere mai la password in chiaro, nemmeno in transito. RAM-USB usa invece deliberatamente un login classico, email e password: esplorare questo schema di autenticazione, e fino a che punto le sue garanzie possano essere spinte, hashing Argon2id salato e pepperato, email cifrata, nessuna persistenza né log in chiaro, è esso stesso oggetto di studio.

La seconda riguarda il trasporto dei backup. Un protocollo a blocchi come iSCSI eviterebbe la maggior parte della complessità che RAM-USB affronta per isolare gli utenti, creazione di utenti POSIX, isolamento chroot, `AuthorizedKeysCommand`, perché il server non dovrebbe mai ragionare su directory o tentativi di fuga. RAM-USB fa invece backup su filesystem via SFTP, perché l'interesse di ricerca è specificamente nell'isolamento a livello di filesystem e nella prevenzione della fuga dalla propria directory tramite chroot.

1.3 Obiettivi e perimetro

RAM-USB è un'architettura a microservizi n-tier, con dieci componenti interni (Capitolo 4). Anche quella cifra, però, non riflette più esattamente la topologia reale: Database-Vault include ormai il proprio PostgreSQL, Metrics-Collector la propria TimescaleDB, e Certificate-Authority il proprio sidecar di metriche, ciascuno nella stessa immagine invece che come processo separato.

Alcune funzionalità restano fuori dal perimetro del progetto, per due ragioni diverse. Sono temporaneamente fuori perimetro, per vincoli di tempo più che per scelta di design: la modifica delle credenziali di un utente, la revoca del suo accesso al sistema, la protezione dei backup da modifica o cancellazione anche da parte di un amministratore, e la conformità GDPR. Sono invece permanentemente fuori perimetro, perché estranee all'oggetto di studio: un'interfaccia grafica per l'utente finale, e ogni limite commerciale o di fatturazione.

1.4 Metodologia

L'implementazione segue un modello a V [2]: ogni livello di specifica ha un corrispondente livello di test, e il test si scrive prima del codice che verifica [3]. Il linguaggio è Go, per l'intero stack lato server e per il client CLI.

Parte dell'implementazione è stata realizzata con l'assistenza di Claude Sonnet 5, usato come strumento di implementazione sotto supervisione stretta [4]. Il confine di responsabilità è netto: la progettazione, la struttura del codice, i confini dei moduli, la firma e il comportamento delle singole funzioni, e i test corrispondenti restano una decisione umana; l'assistente implementa esattamente, e solo, quanto specificato, in passi piccoli e incrementali, e nessuna modifica entra nella codebase senza revisione e approvazione esplicita.

Questo confine non è solo dichiarato come policy, ma imposto anche a livello meccanico, tramite le regole di permesso dell'ambiente di sviluppo: i comandi che scriverebbero direttamente nella cronologia del repository restano bloccati per l'assistente indipendentemente da cosa gli venga chiesto, e ogni commit passa comunque da una revisione umana prima di entrare nel repository.

Due pratiche minori di processo completano la metodologia. I gap di implementazione scoperti nel corso del lavoro sono tracciati separatamente dalle decisioni di perimetro deliberate della SRS: i primi restano in `docs/Known_Issues.md` [5] fino a che non diventano una issue di GitHub o vengono scartati; le seconde restano nella sezione 8 della SRS, quella dei rischi accettati (Capitolo 12).

I diagrammi del progetto, inoltre, sono codice e non immagini statiche: sorgenti PlantUML versionati nel repository, rigenerati a ogni modifica con un unico comando [6].

1.5 Struttura della tesi

Il Capitolo 2 colloca RAM-USB nel panorama degli strumenti e dei principi già esistenti, dal backup cifrato lato client alle reti mesh zero-trust. Il capitolo 3 descrive la metodologia con cui i requisiti sono stati raccolti e tracciati, e i casi d'uso che ne derivano.

Il Capitolo 4 presenta i componenti che partecipano alla registrazione e al login. Il Capitolo 5 segue le credenziali lungo quel percorso, dal confine pubblico fino al record salvato, e descrive le regole che ogni strato applica.

Il Capitolo 6 segue lo stesso backup fino al proprio spazio isolato su Storage-Service. Il Capitolo 7 descrive come la rete mesh applichi i confini di raggiungibilità tra i componenti. Il Capitolo 8 chiude il ciclo di vita di una richiesta con la pipeline di osservabilità che la rende misurabile.

Il Capitolo 9 verifica il sistema così costruito attraverso i quattro livelli del modello a V, dal test di unità alla lista di accettazione, e riporta cosa ha trovato eseguendolo su hardware reale. Il Capitolo 10 descrive come questi componenti siano effettivamente distribuiti, tra container Docker, macchine Proxmox e un VPS pubblico. Il Capitolo 11 rilegge l'intero sistema dal punto di vista di chi tenta di comprometterlo, e ne quantifica i costi.

Stato dell'arte e contesto tecnologico

2.1 Principi di sicurezza di riferimento

Il modello zero-trust adottato da RAM-USB non è un'invenzione del progetto, ma l'applicazione di un principio già formalizzato. NIST Special Publication 800-207 lo definisce come l'assunzione che nessuna rete, interna o esterna, debba essere considerata implicitamente affidabile, e che ogni richiesta di accesso vada valutata indipendentemente dalla sua provenienza [7]. La rivalidazione indipendente ad ogni strato, descritta nel Capitolo 5, è l'applicazione diretta di questo principio dentro i confini di un singolo sistema, non solo al suo perimetro esterno.

Il termine "zero-knowledge" merita invece una distinzione che la letteratura sui prodotti commerciali tende a sfumare. In crittografia, una zero-knowledge proof è un protocollo formale in cui un dimostratore convince un verificatore della verità di un'affermazione senza rivelargli nulla al di là di quella verità [8].

RAM-USB, come i prodotti descritti nelle prossime sezioni, non implementa alcuna zero-knowledge proof: usa "zero-knowledge" nel senso più debole, e più comune nel settore, di "il fornitore del servizio non può accedere ai dati in chiaro". La garanzia che ne deriva è reale, ma è una proprietà architetturale, ottenuta cifrando lato client prima della trasmissione, non una proprietà dimostrata crittograficamente.

2.2 Soluzioni di backup esistenti

RAM-USB non implementa da zero la cifratura, la deduplicazione e la compressione dei backup: le affida a restic [9], già descritto nel Capitolo 6, che le esegue interamente sul dispositivo dell'utente prima che qualunque byte raggiunga Storage-Service.

Restic non è l'unico strumento a offrire queste proprietà. Tarsnap cifra ogni archivio lato client con una chiave AES-256 generata localmente, e invia al server solo le chiavi HMAC-SHA256 necessarie a firmare le richieste, mai le chiavi di cifratura vere e proprie [10].

BorgBackup offre lo stesso insieme di garanzie di restic, cifratura AES-256 lato client, deduplicazione a blocchi a contenuto variabile, compressione, ma con un modello di trasporto più ristretto: il repository remoto è raggiungibile solo via SSH, mentre restic supporta nativamente anche S3, Backblaze B2, Azure Blob Storage e altri backend [11].

Quello che accomuna i tre strumenti, e che RAM-USB eredita scegliendo restic, è la stessa garanzia architetturale della sezione precedente: il server non vede mai il contenuto in chiaro, perché non lo vede mai nessun componente al di fuori del dispositivo dell'utente.

2.3 Paralleli in altri domini a conoscenza zero

La stessa garanzia compare, sotto lo stesso nome, in domini diversi dal backup. Bitwarden, un password manager, dichiara esplicitamente un'architettura zero-knowledge: ogni dato è cifrato sul dispositivo dell'utente prima di raggiungere i suoi server, che custodiscono quindi solo dati già cifrati, mai le chiavi per leggerli [12]. È lo stesso principio che RAM-USB applica alla password dell'utente nel Capitolo 5, sebbene in un dominio diverso: la credenziale di accesso al servizio, non il suo contenuto.

Tresorit applica un principio simile allo storage cloud con sincronizzazione: ogni file è cifrato con una chiave propria, generata e mai trasmessa in chiaro, e anche la condivisione tra utenti avviene tramite crittografia a chiave pubblica, in modo che nemmeno Tresorit possa accedere alle chiavi condivise [13].

A differenza di RAM-USB, però, Tresorit è pensato per la sincronizzazione e la condivisione in tempo reale, non solo per il backup: gestisce permessi granulari e revoca dell'accesso per singolo file, una funzionalità che RU-09 lascia esplicitamente fuori dal perimetro di RAM-USB.

2.4 Reti mesh e zero-trust networking

La rete privata di RAM-USB non è una VPN centralizzata, ma una mesh coordinata da Headscale, un'implementazione self-hosted del control server di Tailscale [14], già introdotta nel Capitolo 4. La differenza non è solo terminologica.

In una VPN centralizzata, ogni pacchetto tra due nodi attraversa un concentratore centrale, che deve quindi restare sul percorso dati di ogni connessione. Tailscale distribuisce invece solo il piano di controllo da un server centrale: ogni nodo genera la propria coppia di chiavi WireGuard e la mantiene locale, mentre il coordinatore si limita a distribuire le chiavi pubbliche e le policy tra i nodi autorizzati.

Quando possibile, tra i due nodi che devono comunicare si stabilisce così un tunnel diretto, cifrato con WireGuard, senza che il coordinatore resti nel percorso dei dati [15]. Le regole di raggiungibilità che Network-Manager pubblica su Headscale (capitolo 7) sfruttano esattamente questa separazione: il coordinatore decide chi può contattare chi, ma non si trova mai sul percorso dei dati che poi vengono scambiati.

2.5 Posizionamento di RAM-USB

Nessuno degli strumenti descritti in questo capitolo è, da solo, un termine di paragone diretto per RAM-USB, e per una ragione precisa: sono prodotti, mentre RAM-USB è un caso di studio. Un prodotto ottimizza per l'utente finale, la compatibilità, la scala; un caso di studio ottimizza per la comprensione della singola scelta di design, anche quando quella scelta costa in prestazioni o in copertura funzionale, come già dichiarato nel Capitolo 1.

RAM-USB riprende da questi strumenti le proprietà, non l'implementazione: la cifratura lato client di restic, Tarsnap e Borg; la garanzia zero-knowledge di Bitwarden e Tresorit, applicata però sia alle

credenziali sia ai file; la topologia a mesh di Tailscale, riletta come confine di raggiungibilità e non solo come trasporto.

Quello che nessuno di questi strumenti affronta insieme è l'isolamento a livello di filesystem tramite chroot dinamico (Capitolo 6) e la rivalidazione indipendente a ogni strato di una catena di quattro processi (Capitolo 5): è precisamente lì che si concentra il contributo di RAM-USB.

Requisiti: metodologia e casi d'uso

3.1 Metodologia dei requisiti

I requisiti di RAM-USB sono raccolti in un documento unico chiamato Software Requirements Specification (SRS) [1], che costituisce l'unica fonte di verità su cosa il sistema debba fare. La scrittura di tale documento segue un modello a spirale dell'ingegneria del software [16], guidato dalla progressiva risoluzione del rischio, inteso come le fonti di incertezza più significative del progetto. Il livello di dettaglio della SRS aumenta di conseguenza ad ogni iterazione. Gli obiettivi e i requisiti di alto livello erano chiari fin dalle fasi iniziali. La conoscenza degli strumenti e delle tecnologie di terze parti, però, non bastava a prevederne con certezza il comportamento a basso livello. La scelta del modello a spirale nasce da questo squilibrio. Molti vincoli concreti sono infatti emersi solo nel momento dell'implementazione, quando l'interazione diretta con questi strumenti ha rivelato dettagli che una fase di analisi puramente teorica, non è stata in grado di anticipare. Un caso concreto di questa dinamica è il requisito NM-F-12, rivisitato in tre round successivi:

- nella versione 1.0 della SRS [17] assumeva che Network-Manager (il componente responsabile della rete privata del sistema) dovesse costruire da zero un proprio meccanismo per autorizzare l'accesso di nuovi dispositivi;

- nella versione 1.4 [18] è emerso che Headscale, lo strumento di coordinamento della rete impiegato dal sistema, offriva già questa funzionalità tramite la propria CLI, eliminando la necessità di scriverla;

- nella versione 1.7 [19] è emerso un limite più profondo dello stesso Headscale (non può coordinare una rete di cui fa parte) approfondito nella sezione 7.3, richiedendo una riformulazione più ampia del requisito.

3.2 Tracciabilità dei requisiti

Un aspetto metodologico distintivo del progetto è la tracciabilità, intesa come vincolo tecnico verificabile in tre direzioni indipendenti. Questo approccio privilegia la semplicità di un audit di sicurezza a scapito della complessità della gestione dell'implementazione.

- **Dal requisito alla sua implementazione:** ogni requisito della SRS che ha già un'implementazione è collegato a un permalink GitHub che punta a un commit specifico e ad un intervallo di righe preciso.
- **Dal test al requisito e viceversa:** ogni funzione di test porta, come commento di documentazione nella riga immediatamente precedente, l'identificativo del requisito che verifica. La relazione non è necessariamente 1-a-1 visto che un requisito può avere più test che lo verificano. Inoltre, la tracciabilità vale in entrambe le direzioni: si può infatti passare dal requisito ai test, cercando l'ID nella codebase; e dal test al requisito, leggendo il commento sovrastante la funzione.
- **Dal commit al requisito:** ogni commit del repository segue una grammatica fissa, descritta nel `CONTRIBUTING.md` [20] e qui sintetizzata come tipo, area, descrizione del cambiamento, seguiti dall'elenco degli identificativi dei requisiti che copre. Come per il collegamento test-requisito, anche questo vale in entrambe le direzioni: dal requisito, se ne cerca l'ID nella storia del repository; nella direzione opposta, l'elenco finale della descrizione di ogni commit indica subito quali requisiti sono coperti.

3.3 Requisiti utente e casi d'uso

I requisiti utente (RU) esprimono, dal punto di vista di chi usa il sistema, cosa RAM-USB deve garantire. Sono gli unici requisiti riportati integralmente nel corpo della tesi; i requisiti di sistema che li realizzano (funzionali, non funzionali e di dominio) sono troppo numerosi per un elenco completo, e sono raccolti in Appendice A.

RU-01 Registrarsi al servizio fornendo la quantità minima di dati necessaria.

RU-02 Potersi autenticare in sessioni successive alla registrazione.

RU-03 Poter caricare backup cifrati verso il servizio da qualsiasi luogo.

RU-04 Poter accedere al proprio backup solo dopo essersi autenticato.

RU-05 Avere la garanzia che nessuno, amministratori inclusi, possa leggere i propri backup.

RU-06 Avere la garanzia che nessuno, amministratori inclusi, possa leggere le proprie credenziali di accesso.

RU-07 Poter recuperare i propri file in qualsiasi momento.

RU-08 Avere la certezza che i propri dati siano isolati da quelli degli altri utenti.

RU-09 (*Fuori perimetro*) Che nessuno, amministratori inclusi, possa modificare o cancellare i propri backup.

RU-10 (Amministratore) Poter osservare la salute e le prestazioni del sistema senza compromettere i dati degli utenti.

I requisiti utente si traducono nei seguenti casi d'uso, illustrati anche nella Figura 3.1

UC-01 (Registrazione) Un nuovo utente fornisce email, password e chiave pubblica SSH, e ottiene un account attivo.

UC-02 (Autenticazione) Un utente registrato fornisce email e password, e ottiene un accesso temporaneo al proprio spazio di backup.

UC-03 (Backup di un file) Un utente autenticato carica un file, già cifrato sul proprio dispositivo, nel proprio spazio isolato.

UC-04 (Restore di un file) Un utente autenticato recupera un proprio file precedentemente caricato, decifrandolo localmente.

UC-05 (Consultazione delle metriche operative) Un amministratore consulta lo stato di salute e le prestazioni del sistema, senza accedere ai dati dei singoli utenti.

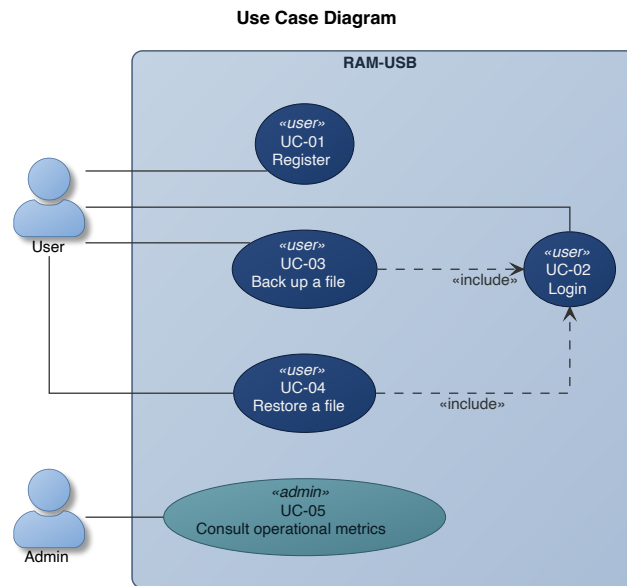


Figura 3.1: Diagramma dei casi d'uso principali di RAM-USB.

I capitoli successivi descrivono come l'architettura del sistema realizzi questi casi d'uso: il capitolo 4 introduce la struttura generale dei componenti e i confini di sicurezza tra di essi, mentre i capitoli 5-8 approfondiscono ciascun dominio funzionale nel dettaglio.

Architettura generale e trust zones

4.1 Overview architetturale

RAM-USB è un'architettura a microservizi n-tier in cui ogni componente ha una responsabilità specifica. Questa separazione riduce il rischio che la compromissione di un componente comprometta l'intero sistema.

RAM-USB ha dieci componenti interni, organizzati in tre gruppi funzionali:

- **Accesso e registrazione:** Entry-Hub, Security-Switch, Database-Vault, Network-Manager, Certificate-Authority, Headscale.
- **Backup e restore:** Storage-Service.
- **Osservabilità:** Mosquitto, Metrics-Collector, Grafana.

La Figura 4.1 mostra questi dieci componenti e il protocollo con cui interagiscono.

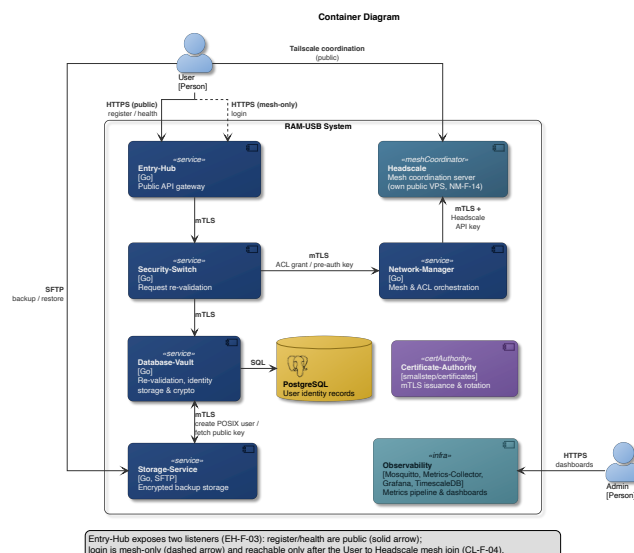


Figura 4.1: Diagramma dei container di RAM-USB.

4.1.1 Entry-Hub

Entry-Hub è il gateway pubblico del sistema, ovvero l'unico esposto a chiunque su internet. Per questo il sistema non gli concede fiducia implicita. Il suo processo gira quindi in un container distroless [21], seguendo il principio di riduzione dell'attack surface raccomandato da NIST SP 800-190 [22]. Senza shell, infatti, la maggior parte delle vulnerabilità critiche note nei container non è sfruttabile [23].

Entry-Hub espone tre endpoint HTTPS: `GET /api/health` e `POST /api/users` sono raggiungibili da chiunque su internet, con certificato firmato dalla CA pubblica Let's Encrypt [24]. `POST /api/login` usa lo stesso certificato, ma ha un listener separato, raggiungibile solo da utenti con accesso alla rete privata, descritta in dettaglio nel Capitolo 7.

Quando riceve una richiesta, Entry-Hub valida il corpo JSON in due fasi: in decodifica, impone un limite di dimensione e rifiuta qualunque campo non atteso nel payload; poi valida i singoli campi email, password e, solo in fase di registrazione, la chiave SSH [25]. Se una di queste verifiche fallisce, risponde HTTP 400 senza specificare quale sia il problema e senza inoltrare la richiesta agli strati più interni del sistema. Se la validazione riesce, inoltra la richiesta a Security-Switch via mTLS verificando che il certificato ricevuto appartenga davvero a un Security-Switch. Attende la risposta del Security-Switch e la rilancia al client.

4.1.2 Security-Switch

Security-Switch esiste per evitare che le responsabilità di contattare Database-Vault e Network-Manager siano in mano al punto di ingresso del sistema: riceve le richieste in ingresso solo da Entry-Hub, autenticato via mTLS, le rivalida in modo indipendente e contatta i servizi interni.

Lo scopo della rivalidazione è l'applicazione del principio di defence in depth [7]. La logica di validazione, gestione degli errori e logging viene riutilizzata da tutti i servizi principali e vive in librerie interne condivise [26].

Se la validazione riesce, inoltra la richiesta a Database-Vault via mTLS.

Nel caso di una richiesta di registrazione, chiede al Network-Manager di creare l'identità del nuovo utente sulla rete mesh e la credenziale (pre-auth key) con cui il suo dispositivo potrà unirvisi.

Nel caso di una richiesta di login, chiede al Network-Manager di concedere un accesso temporaneo di 12 ore per quell'utente verso Storage-Service. Solo quando l'utente avrà ottenuto l'accesso allo Storage-Service, potrà contattarlo e gestire i propri backup.

4.1.3 Database-Vault

Database-Vault è il componente che si occupa di memorizzare le credenziali degli utenti in un database PostgreSQL. È l'unico in grado di accedere al database, ma non ha tuttavia accesso ai file degli utenti, i quali si trovano persistiti sullo Storage-Service. Questa separazione limita l'impatto della compromissione di un singolo componente.

Il diagramma Entità-Relazioni è illustrato in Figura 4.2. Visto che il database deve contenere solo i dati degli utenti e non vi è alcuna relazione significativa da salvare, l'ER è piuttosto semplice.

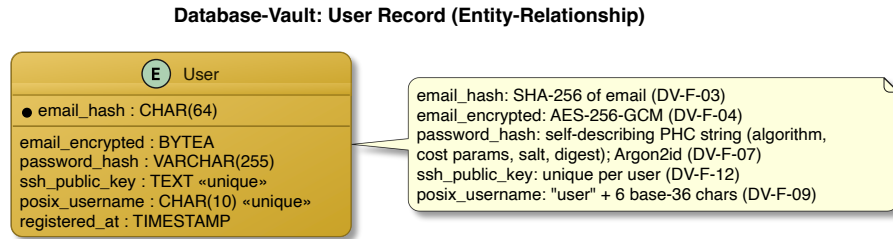


Figura 4.2: Diagramma Entità-Relazioni del record utente in Database-Vault.

L'email è cifrata, recuperabile solo con la master key, mentre la password non è mai recuperabile, poiché viene hashata con Argon2id. Il furto o l'accesso non autorizzato al database non bastano quindi a leggere né l'una né l'altra.

In fase di registrazione di un nuovo utente, in seguito alla rivalidazione dell'input, Database-Vault tenta di salvare i dati in una transazione atomica, rifiutando la richiesta se l'email o la chiave SSH sono già registrate, ma senza specificare quale delle due sia errata. Se il salvataggio riesce, richiede a Storage-Service la creazione dello spazio POSIX dell'utente. In caso di fallimento dello Storage-Service, elimina il record appena creato, altrimenti informa Security-Switch che la registrazione è andata a buon fine.

Al login, ricalcola l'hash della password ricevuta e lo confronta con quello salvato, poi risponde a Security-Switch.

Un'email inesistente e una password sbagliata producono la stessa risposta: in questo modo, chi tenta l'accesso non può usarla per scoprire quali email sono registrate. Il Capitolo 11, tuttavia, spiega come un utente esterno può tentare di capire se un'email è già registrata nel sistema e anche come un amministratore malintenzionato può ottenere le credenziali di accesso di un utente.

4.1.4 Storage-Service

Storage-Service ospita i file di backup degli utenti. Non elabora mai contenuto in chiaro poiché per design, non è in grado di farlo. Riceve infatti soltanto file deduplicati, compressi e cifrati lato client con Restic [9].

Su richiesta di Database-Vault, crea un utente POSIX dedicato, senza shell interattiva né home directory tradizionale. Ogni utente ha infatti uno spazio scrivibile in una sua sottodirectory dentro un chroot. La struttura del file system è illustrata in Figura 4.3.

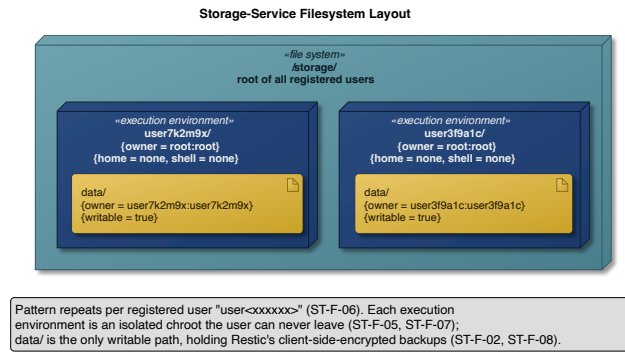


Figura 4.3: Struttura del filesystem di Storage-Service.

L'accesso allo Storage-Service può avvenire solo via SFTP [27], con la chiave pubblica SSH inviata dall'utente in fase di registrazione. Non sono quindi permessi né l'autenticazione con username e password, né altre forme di accesso SSH al di fuori della SFTP.

Ad ogni connessione, un binario `AuthorizedKeysCommand` dedicato interroga Database-Vault per ottenere la chiave pubblica corrente dell'utente. Qualunque errore in quella chiamata, nega la connessione secondo il principio fail-secure [28].

4.1.5 Network-Manager

Network-Manager è l'amministratore delle regole di rete. Si occupa di definire, modificare ed eliminare le regole che permettono ad un utente della rete privata mesh di raggiungere gli altri membri.

In fase di registrazione, crea un utente Headscale dedicato e la pre-auth key con cui il dispositivo dell'utente si unirà alla rete.

Durante il login, assegna al nodo dell'utente il tag ACL (Access Control List) che abilita l'accesso temporaneo di 12 ore allo Storage-Service. Un processo periodico controlla le scadenze e rimuove il tag dai nodi scaduti.

4.1.6 Certificate-Authority

Certificate-Authority è la CA privata del sistema, basata su `smallstep/certificates` [29].

Emette, rinnova e revoca i certificati mTLS di ogni servizio interno, il che lo rende un componente particolarmente critico del sistema, nonché l'unico raggiungibile da tutti i servizi interni: un malintenzionato che riuscisse a comprometterla, infatti, potrebbe emettere certificati falsificati per qualunque organizzazione.

Ogni servizio ottiene il suo primo certificato contattando la CA sfruttando un token di bootstrap distribuito manualmente da un operatore fuori banda. Una spiegazione dettagliata si trova nella sezione 4.4.

4.1.7 Headscale

Headscale è un'implementazione self-hosted del Tailscale control server [14].

Svolge il ruolo di coordinatore della rete privata mesh: ad ogni tentativo di connessione tra due nodi, fa rispettare le regole di raggiungibilità decise da Network-Manager. Distribuisce inoltre, tramite

MagicDNS, i nomi host con cui i nodi si risolvono reciprocamente, permettendo al client di individuare Storage-Service senza conoscerne l'indirizzo IP.

Valida inoltre la pre-auth key di un nuovo dispositivo al momento dell'ingresso nella mesh.

A differenza degli altri componenti interni, fatta eccezione per Entry-Hub, Headscale non risiede nella rete privata, ma è raggiungibile da chiunque su internet. La sezione 7.3 ne approfondisce le ragioni.

4.1.8 Mosquitto

Mosquitto è il broker MQTT su cui ogni servizio, una volta al minuto, pubblica le proprie metriche operative.

Applica una ACL secondo la quale ogni servizio può solo scrivere sul proprio topic, mai leggerlo, mentre Metrics-Collector può solo leggere l'intero spazio `metrics/*`, senza mai scrivervi.

4.1.9 Metrics-Collector

Metrics-Collector è il componente che riceve, valida e rende persistenti le metriche operative pubblicate da ogni servizio.

Scarta ogni metrica ricevuta il cui campo `service` non corrisponde al topic da cui arriva e persiste le metriche accettate come hypertable in TimescaleDB [30], con ritenzione di trenta giorni e compressione dopo sette giorni.

4.1.10 Grafana

Grafana è la piattaforma di visualizzazione con cui un amministratore consulta lo stato di salute e le prestazioni del sistema [31].

Legge da TimescaleDB direttamente, senza passare per Metrics-Collector. La dashboard di Grafana mostra tempi di risposta, throughput e connessioni attive. Il Capitolo 8 tratta questa pipeline di osservabilità nel dettaglio.

4.2 Modello di comunicazione inter-servizio

Ogni servizio espone la propria API HTTPS sulla libreria standard di Go, `net/http`. Le richieste e le risposte sono corpi JSON ed ogni endpoint è registrato su un router minimale, `http.ServeMux`. La versione di TLS ammessa è la 1.3.

Gli endpoint qui elencati seguono le convenzioni REST [32].

- GET `/api/health`, POST `/api/users` di Entry-Hub,
- GET `/internal/v1/public-key/{posix_username}` di Database-Vault,
- POST `/internal/v1/posix-users` di Storage-Service e
- POST `/internal/v1/mesh-users`, POST `/internal/v1/grants` di Network-Manager

POST `/api/login` è l'unica eccezione, perché un login riuscito restituisce solo la conferma che un'azione è avvenuta ma manca l'identificativo che una vera risorsa `sessions` richiederebbe per essere letta con GET o cancellata con DELETE. Il token ACL di 12 ore che il login concede, infatti, resta interno a Network-Manager e scade autonomamente.

La pubblicazione delle metriche inoltre, è asincrona: ogni servizio, infatti, pubblica i propri messaggi su un topic MQTT dedicato, e non c'è alcun bisogno di attendere una risposta.

La struttura del sistema ha un costo non indifferente: ogni registrazione e ogni login attraversano quattro processi distinti, ognuno con il proprio handshake mTLS e la propria rivalidazione. La latenza è quindi ben più alta rispetto ad un sistema monolitico, e non è possibile ridurla saltando uno dei passaggi qualora fosse necessario. Il servizio, inoltre, resta disponibile agli utenti solo finché restano disponibili i sette componenti che partecipano al percorso di una richiesta, mentre la catena di osservabilità può cadere senza che l'utente se ne accorga.

I due costi non sono però dello stesso tipo. La latenza è definitiva, e si accetta insieme all'architettura; la disponibilità è invece un problema di deployment, e il Capitolo 10 discute fino a che punto l'infrastruttura sottostante possa attenuarlo. Il Capitolo 11 riprende invece il bilancio complessivo, mettendo accanto a ogni costo la proprietà che compra.

4.3 Trust zones e confini di sicurezza

La Figura 4.4 mostra le regole di raggiungibilità che Network-Manager applica alla rete mesh.

Le due zone più esterne appartengono all'utente, prima e dopo l'autenticazione. Un utente registrato ma non ancora autenticato, che appartiene quindi alla *Unauthenticated User Zone* vede e può contattare solo Entry-Hub. Dopo il login, lo stesso nodo entra nella *Authenticated User Zone* e ottiene la raggiungibilità temporanea verso lo Storage-Service.

Solo la Certificate-Authority ha un accesso davvero universale: raggiunge, ed è raggiunta da ogni altro componente interno, per permettergli di rinnovare il proprio certificato X.509 senza un intermediario.

La quarta zona, *Public Zone*, contiene soltanto Headscale, il quale è raggiungibile dalla rete pubblica. La sezione 7.3 ne spiega il motivo nel dettaglio. Tuttavia, le chiamate di Network-Manager verso Headscale sono comunque protette dall'autenticazione mTLS o dal token di bootstrap.

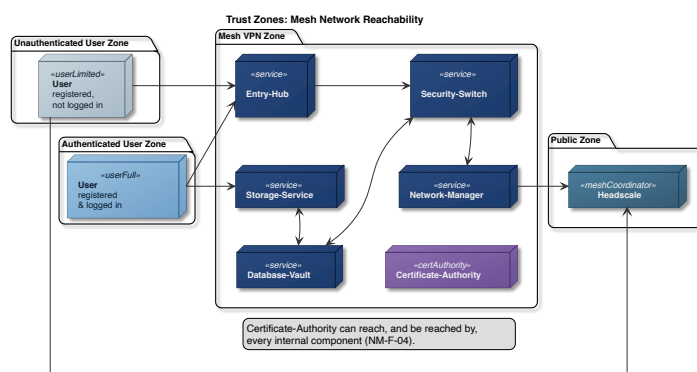


Figura 4.4: Trust zones e raggiungibilità sulla rete mesh.

4.4 PKI e gestione dei certificati

RAM-USB usa due autorità di certificazione, come illustrato in Figura 4.5.

Let's Encrypt, una CA pubblica, firma soltanto il certificato che Entry-Hub presenta sui propri endpoint pubblici. Questo perché il certificato deve essere verificabile da un client esterno che non ha

ancora accesso alla rete interna del sistema. Un certificato firmato dalla Certificate-Authority privata di RAM-USB non sarebbe attendibile.

Ogni altra comunicazione usa invece una Certificate-Authority privata, basata su `smallstep/certificates`.

Ogni certificato privato segue lo stesso schema di bootstrap: ogni servizio riceve, fuori banda e una sola volta, un token di bootstrap monouso, e lo presenta alla Certificate-Authority in cambio del proprio certificato iniziale. I rinnovi successivi presentano il certificato mTLS già posseduto. Il token resta un segreto usa-e-getta e non necessita quindi di essere protetto una volta consumato.

Un certificato X.509 valido dimostra solo che la Certificate-Authority ha approvato quel client, non che sia il client atteso per quella connessione. Per questo ogni servizio verifica anche il campo `organization` del certificato.

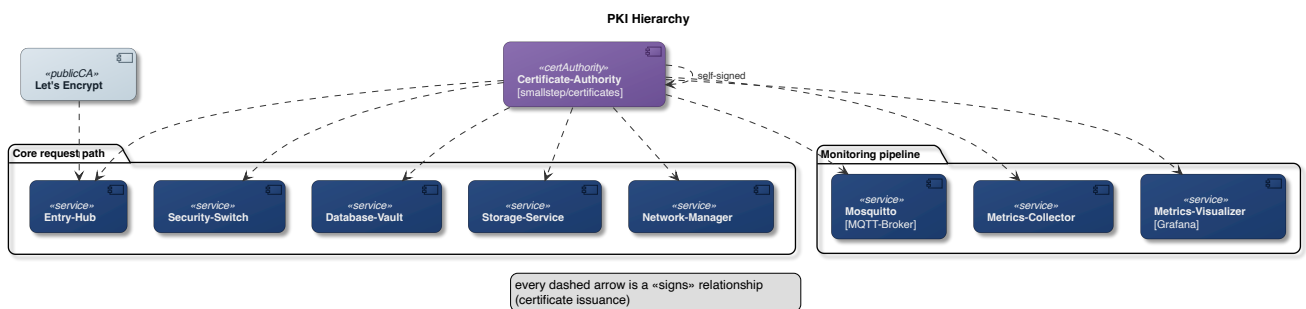


Figura 4.5: Gerarchia delle Certificate Authority e relazioni di firma.

Autenticazione e gestione delle identità

5.1 Ciclo di vita della richiesta

Una richiesta di registrazione o di login attraversa tre servizi prima di diventare un record: Entry-Hub, Security-Switch e Database-Vault. Tutti e tre applicano le stesse verifiche sull'input, ognuno indipendentemente dagli altri.

La validazione avviene in due fasi. La prima riguarda il corpo della richiesta: il payload viene letto fino a un limite di 2048 byte e rifiutato se lo supera, senza accumulare in memoria dati controllati da chi ha inviato la richiesta, e il decoder JSON respinge qualunque campo inatteso.

La seconda fase riguarda i singoli campi [25]. L'email deve essere un indirizzo singolo e sintatticamente valido secondo RFC 5322 [33]; la password deve essere lunga tra 8 e 128 caratteri e usare almeno tre categorie tra minuscole, maiuscole, cifre e simboli. In registrazione, inoltre, la chiave pubblica SSH deve essere ben formata.

Se una verifica fallisce, la risposta è HTTP 400 senza indicare quale controllo non sia passato, il log registra il problema senza identificare l'utente, e la richiesta non prosegue verso l'interno. Il messaggio resta generico perché un errore dettagliato aiuterebbe un attaccante più di quanto aiuti un utente in buona fede, che le stesse verifiche le ha già ricevute dal proprio client.

I tre servizi condividono la stessa libreria di validazione, quindi ripetere il controllo non introduce diversità algoritmica, ma garantisce che nessun servizio dia per scontato che il precedente abbia fatto il proprio lavoro. In questo modo, se a Database-Vault dovesse arrivare una richiesta proveniente da un servizio compromesso, la validazione verrebbe fatta comunque.

Gli errori interni vengono infine tradotti in codici HTTP. La Tabella 5.1 riassume i codici che raggiungono l'utente, attribuendo ciascun codice a chi lo emette.

Entry-Hub costruisce soltanto gli errori che nascono da sé stesso: una validazione fallita e una chiamata a Security-Switch andata male. Tutto il resto lo rilancia al client così come gli arriva dall'interno, senza modificarlo. Per questo motivo, un 401 o un 409 raggiungono l'utente pur non essendo mai prodotti da Entry-Hub.

Tabella 5.1: Codici HTTP restituiti al client e loro significato nel sistema.

Codice	Significato	Emesso da
400	Validazione fallita, senza indicare quale controllo non sia passato	entrambi
401	Credenziali non valide, non specifica se email inesistente o password errata	Security-Switch
403	Rifiuto esplicito di Network-Manager	Security-Switch
409	Email o chiave SSH già registrate, non specifica quale delle due	Security-Switch
500	Errore interno non altrimenti classificato	entrambi
502	Servizio a valle irraggiungibile o con risposta inattesa	entrambi
503	Chiamata a Security-Switch scaduta	Entry-Hub
504	Chiamata a Database-Vault o Network-Manager scaduta	Security-Switch

Tra i codici che i due servizi costruiscono per conto proprio ci sono due differenze: la prima è il 403, che solo Security-Switch può emettere, perché è l'unico dei due a poter ricevere un rifiuto esplicito da un servizio interno. La seconda riguarda le chiamate scadute: Entry-Hub risponde 503, Security-Switch 504.

In tutti i casi il corpo della risposta è sanificato, e il dettaglio dell'errore resta nei log interni.

5.2 Cifratura e hashing delle credenziali

Database-Vault tratta i dati che riceve in tre modi diversi:

- la chiave SSH è pubblica, quindi non necessita di segretezza;
- l'email è insieme una chiave di ricerca e un dato personale da proteggere, ma deve anche essere recuperabile in caso di necessità
- la password non deve essere recuperabile, nemmeno da chi amministra il database.

L'email viene normalizzata in minuscolo e ridotta al proprio digest SHA-256, che diventa la chiave primaria del record. La normalizzazione garantisce che lo stesso indirizzo scritto con lettere diverse al login produca la stessa chiave della registrazione. Questo hash è deterministico per necessità, perché deve permettere il lookup. Il limite fondamentale di questo approccio, insieme alla sua mitigazione, è discusso nel Capitolo 11.

Accanto al digest che fa da chiave di ricerca, il record conserva una copia cifrata, così che l'indirizzo resti recuperabile. Da una master key e da un salt casuale di 16 byte, generato per quel record, viene derivata con HKDF-SHA256 [34] una chiave dedicata, con cui l'email è cifrata in AES-256-GCM [35] usando un nonce casuale di 12 byte. Salt, nonce e testo cifrato vengono salvati nel record, mentre la chiave derivata viene azzerata appena il cifrario l'ha consumata. Derivare una chiave per ogni record, invece di cifrare tutto con la master key, riduce la quantità di dati protetti da una singola chiave.

```

func newGCM(masterKey, salt []byte) (cipher.AEAD, error) {
    derivedKey := make([]byte, derivedKeySize)
    kdf := hkdf.New(sha256.New, masterKey, salt, []byte(hkdfInfo))
    if _, err := io.ReadFull(kdf, derivedKey); err != nil {
        return nil, fmt.Errorf("encryption: derive key: %w", err)
    }

    block, err := aes.NewCipher(derivedKey)
    for i := range derivedKey {
        derivedKey[i] = 0
    }
    if err != nil {
        return nil, fmt.Errorf("encryption: new AES cipher: %w", err)
    }

    gcm, err := cipher.NewGCM(block)
    if err != nil {
        return nil, fmt.Errorf("encryption: new GCM: %w", err)
    }

    return gcm, nil
}

```

Codice 5.1: Derivazione della chiave per record in `newGCM`, da `encryption.go` [36].

La password viene invece concatenata a un pepper, letto da una variabile d'ambiente, e passata ad Argon2id [37] insieme a un salt casuale di 16 byte generato sul momento per quel record.

OWASP [38] propone diverse configurazioni equivalenti tra loro, che scambiano memoria e numero di passaggi. RAM-USB parte da quella con più memoria e ne raddoppia i passaggi, pagando quindi a ogni verifica il doppio del lavoro che OWASP considera sufficiente. È un costo che rende impraticabile recuperare le password da un database rubato. Il pepper complica ulteriormente un attacco offline: a differenza del salt, che è salvato nel record e impedisce soltanto che un tentativo valga per più utenti insieme, il pepper nel database non compare: in sua assenza nessun tentativo è calcolabile, e una copia del database non è quindi sufficiente a condurre l'attacco.

Il risultato viene salvato come stringa autodescrittiva in formato PHC, la convenzione di codifica nata dalla Password Hashing Competition. Ogni record porta quindi con sé i parametri con cui è stato creato, e la verifica al login li usa al posto delle costanti correnti del servizio. In questo modo, cambiare i parametri per i nuovi utenti non invalida i record già esistenti.

La Tabella 5.2 elenca i campi di quella stringa, nell'ordine in cui compaiono nel record.

Tabella 5.2: Campi della stringa PHC in cui Database-Vault salva l'hash della password.

Campo	Significato
argon2id	Variante dell'algoritmo
v=19	Versione di Argon2: il byte 0x13 fissato da RFC 9106, stampato in decimale
m=47104	Memoria di lavoro in KiB, cioè 46 MiB
t=2	Numero di passaggi sulla memoria
p=1	Grado di parallelismo
salt	Salt casuale di 16 byte, generato per quel record
digest	Hash di 32 byte prodotto da Argon2id

Master key e pepper risiedono entrambi in variabili d'ambiente, senza una procedura di backup né di rotazione. Il rischio è discusso nel Capitolo 12.

Lo Schema 5.1 riassume le trasformazioni appena descritte, con i simboli seguenti:

- e : l'email ricevuta;
- p : la password ricevuta;
- k_{SSH} : la chiave pubblica SSH;
- π : il pepper;
- MK : la master key;
- s, n, s_p : rispettivamente il salt di HKDF, il nonce di AES-GCM e il salt di Argon2id, generati per quel record;
- $\xleftarrow{R}$: un'estrazione casuale uniforme dall'insieme indicato.

1. $s, s_p \xleftarrow{R} \{0, 1\}^{128}, \quad n \xleftarrow{R} \{0, 1\}^{96}$
2. $k_h = \text{SHA-256}(\text{lower}(e))$
3. $k = \text{HKDF-SHA256}(MK, s)$
4. $c = \text{AES-GCM}_k(n, e)$
5. $h = \text{Argon2id}(p \parallel \pi, s_p)$
6. $r = (k_h, s, n, c, \text{PHC}(h, s_p), k_{SSH})$

Schema 5.1: Trasformazioni applicate alle credenziali prima della scrittura del record.

5.2.1 Robustezza della password

Le regole di validazione della sezione 5.1 delimitano lo spazio delle password ammesse, e da quello spazio si ricava, nel caso migliore, quanta incertezza la policy possa garantire a chi tenta di indovinare una password, ossia quante possibilità debba considerare, non sapendo quale sia stata scelta.

Quanto valga quell'incertezza dipende però da come la password è stata scelta. Nel caso migliore, in cui è prodotta da un generatore pseudo-casuale che pesca dall'intero alfabeto ammesso, ogni password ammessa è equiprobabile e l'incertezza è quindi massima. Nel caso peggiore, in cui è costruita da una persona, alcune password sono ben più probabili di altre, ad esempio, **Password1!** viene scelta molto più

spesso di una sequenza casuale della stessa lunghezza. Il valore calcolato qui è quello del caso migliore, cioè un limite superiore.

In questo caso, quindi, il calcolo si riduce a contare i valori distinti che la policy ammette, a partire dalla cardinalità delle quattro categorie:

$$\begin{aligned}
 n_1 &= 26 && \text{minuscole} \\
 n_2 &= 26 && \text{maiuscole} \\
 n_3 &= 10 && \text{cifre} \\
 n_4 &= 33 && \text{simboli} \\
 N &= \sum_i n_i = 95 && \text{caratteri disponibili in totale}
 \end{aligned}$$

Il vincolo di almeno tre categorie esclude le stringhe che ne usano al più due, quindi per contare le password valide conviene contare quelle che non lo sono: si parte da tutte le N^L stringhe possibili e si sottraggono quelle che usano una sola categoria e quelle che ne usano esattamente due.

Il primo gruppo è immediato, perché le stringhe scritte con la sola categoria i sono n_i^L . Per il secondo non basta sommare $(n_i + n_j)^L$ su tutte le coppie, perché quel valore conta le stringhe scritte con quei due soli alfabeti e comprende quindi anche quelle di sola i e di sola j : sottraendo n_i^L e n_j^L restano le stringhe che usano davvero entrambe le categorie. Per il principio di inclusione-esclusione, il numero di password valide di lunghezza L vale quindi

$$V(L) = N^L - \sum_{i=1}^4 n_i^L - \sum_{i < j} \left[(n_i + n_j)^L - n_i^L - n_j^L \right].$$

Un attaccante privo di qualsiasi informazione deve considerare tutti i $V(L)$ candidati ammessi. Quel numero si esprime abitualmente in bit, cioè come esponente di 2: b bit corrispondono a 2^b candidati. Se la password è estratta uniformemente, l'incertezza dell'attaccante vale perciò $\log_2 V(L)$ bit.

Per la lunghezza minima ammessa, $L = 8$, si ottiene $V(8) = 6,27 \cdot 10^{15}$, cioè $\log_2 V(8) \approx 52,48$ bit.

Il confronto con lo spazio non vincolato mostra quanto pesi ciascuna delle due regole. Senza il vincolo sulle categorie sarebbero ammesse tutte le stringhe di otto caratteri sull'alfabeto, cioè $N^8 = 95^8 = 6,63 \cdot 10^{15}$, cioè 52,56 bit. La suddetta regola quindi riduce i bit di 0,08. Aggiungere un carattere in più, invece, aumenta i bit di entropia di $\log_2 95 \approx 6,57$ bit per ogni carattere aggiunto.

Da queste cifre potrebbe sembrare che il vincolo semplifichi il lavoro di un malintenzionato invece di contrastarlo, ed è vero, per quanto in misura trascurabile, nel caso in cui le password siano create da generatori pseudo-casuali. Lo scopo di quella regola però non è aumentare l'entropia, bensì vincolare la scelta di una persona, che tende a fermarsi sulle sole minuscole o su lettere e cifre. Quanto valga in quel ruolo non è ricavabile da questo calcolo.

Se le due regole debbano coesistere, o se convenga sostituire il vincolo sulle categorie con una lunghezza minima maggiore, resta quindi una questione aperta, ripresa tra gli sviluppi futuri nel Capitolo 12.

Quanto quello spazio resista dipende però anche dal costo di ogni tentativo, cioè dai parametri di Argon2id: il Capitolo 11 quantifica quel costo dal punto di vista di chi attacca.

5.3 Registrazione

Quando la richiesta raggiunge Database-Vault ed è stata rivalidata, il servizio calcola le tre trasformazioni della sezione precedente e prova a scrivere il record in una singola transazione atomica [39]. Due vincoli di unicità proteggono la tabella, uno sull'hash dell'email e uno sulla chiave pubblica SSH: se uno dei due viene violato, la registrazione è rifiutata con HTTP 409, senza dire quale dei due abbia causato il conflitto. Quel codice resta però un'informazione: dice a un mittente non autenticato che uno dei due valori inviati era già presente, e chi invia una chiave SSH appena generata sa che l'unico altro candidato è l'email. Il Capitolo 11 analizza questo problema e presenta una possibile mitigazione.

A questo punto il record esiste, ma l'utente non ha ancora uno spazio in cui scrivere. Database-Vault genera allora un nome utente POSIX nella forma `user<xxxxxx>`, dove le sei cifre sono caratteri casuali di un alfabeto base-36, e chiede a Storage-Service di creare l'account corrispondente. Come Storage-Service lo crei e lo isoli è oggetto del Capitolo 6.

Se quella creazione fallisce, il record appena scritto viene eliminato. Se anche il rollback fallisce, il servizio lo segnala con un errore dedicato.

Quando invece entrambi i passi riescono, Database-Vault comunica a Security-Switch che la registrazione è andata a buon fine, e quest'ultimo procede con l'orchestrazione descritta nella sezione conclusiva di questo capitolo.

```
rollbackCtx, cancel := context.WithTimeout(context.WithoutCancel(ctx), rollbackTimeout)
defer cancel()

if delErr := store.DeleteUser(rollbackCtx, input.EmailHash); delErr != nil {
    return Result{
        Outcome: OutcomeFailed,
        Err: fmt.Errorf("%w: posix creation error: %w; delete error: %w", ErrOrphanedRecord,
            err, delErr),
    }
}
```

Codice 5.2: Rollback compensativo dopo un fallimento di Storage-Service, da `registration.go` [40].

La Figura 5.1 riassume l'intera sequenza, dalla richiesta del client fino alla risposta. Gli ultimi due scambi, quelli verso Network-Manager, sono oggetto della sezione conclusiva di questo capitolo.

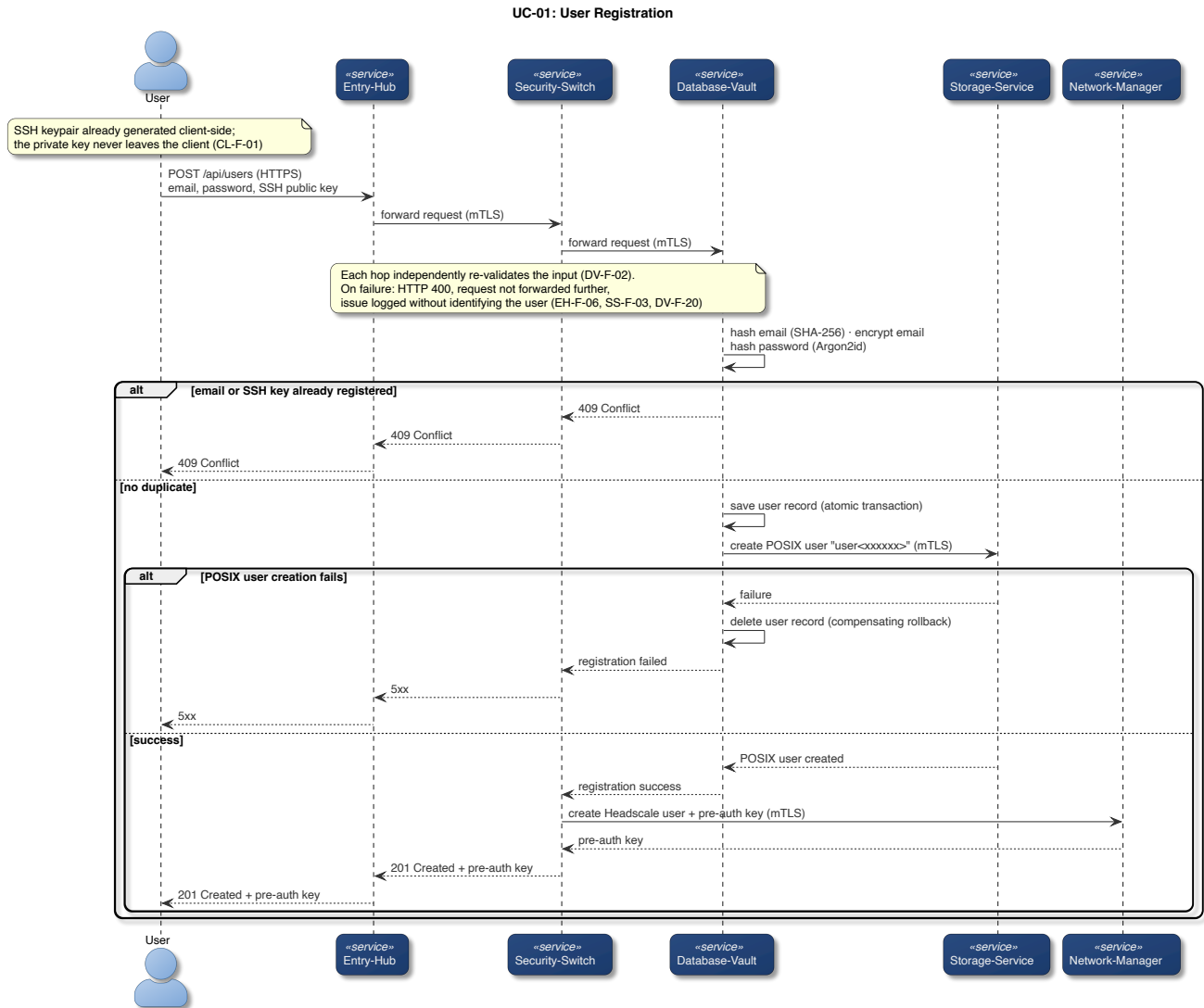


Figura 5.1: Sequenza di registrazione (UC-01).

5.4 Login

Durante il login, Database-Vault ricalcola il digest SHA-256 dell'email ricevuta e con quello cerca il record.

Salt e parametri di costo vengono estratti dalla stringa PHC del record. Argon2id viene ricalcolato con quei valori e con il pepper, e il digest ottenuto è confrontato con quello salvato tramite un confronto a tempo costante, che non lascia quindi dedurre quanti byte iniziali dei due valori coincidano.

Un'email inesistente e una password sbagliata devono produrre la stessa risposta, e non devono essere distinguibili nemmeno nei log. Le due strade convergono quindi su un unico errore sentinella, e il contenuto dell'errore originale viene scartato invece che riportato [41].

Rendere identici la risposta e il log non basta però a rendere indistinguibili i due casi, perché resta una terza differenza osservabile: il tempo. Argon2id, con i parametri della sezione precedente, costa decine di millisecondi, e se venisse eseguito solo quando un record esiste davvero, un attaccante potrebbe

distinguere le due situazioni misurando la latenza della risposta. La soluzione adottata è spendere lo stesso tempo anche quando non c'è nulla da verificare.

```
func VerifyDummy(password, pepper []byte) {
    _, _ = VerifyPassword(password, pepper, dummyHash())
}
```

Codice 5.3: Verifica fittizia usata sui percorsi di rifiuto, da `hash.go` [42].

La funzione ricalcola Argon2id contro un hash fittizio, costruito una sola volta per processo con gli stessi parametri di costo dei record veri, e ne scarta il risultato. Viene chiamata su ogni percorso di rifiuto, compreso quello in cui è il lookup stesso a fallire, così nemmeno un guasto dell'infrastruttura diventa facilmente misurabile dall'esterno.

Ogni incertezza porta allo stesso esito. Se la verifica non può concludersi, per esempio perché l'hash salvato è malformato, la risposta resta 401, secondo il principio fail-secure [28]. Nei log quel caso resta distinguibile, perché a un operatore serve sapere che si tratta di un record corrotto, di un utente distratto o di un vero e proprio attacco.

La Figura 5.2 riassume la sequenza completa di login, compreso il ramo in cui le due cause di rifiuto convergono sulla stessa risposta.

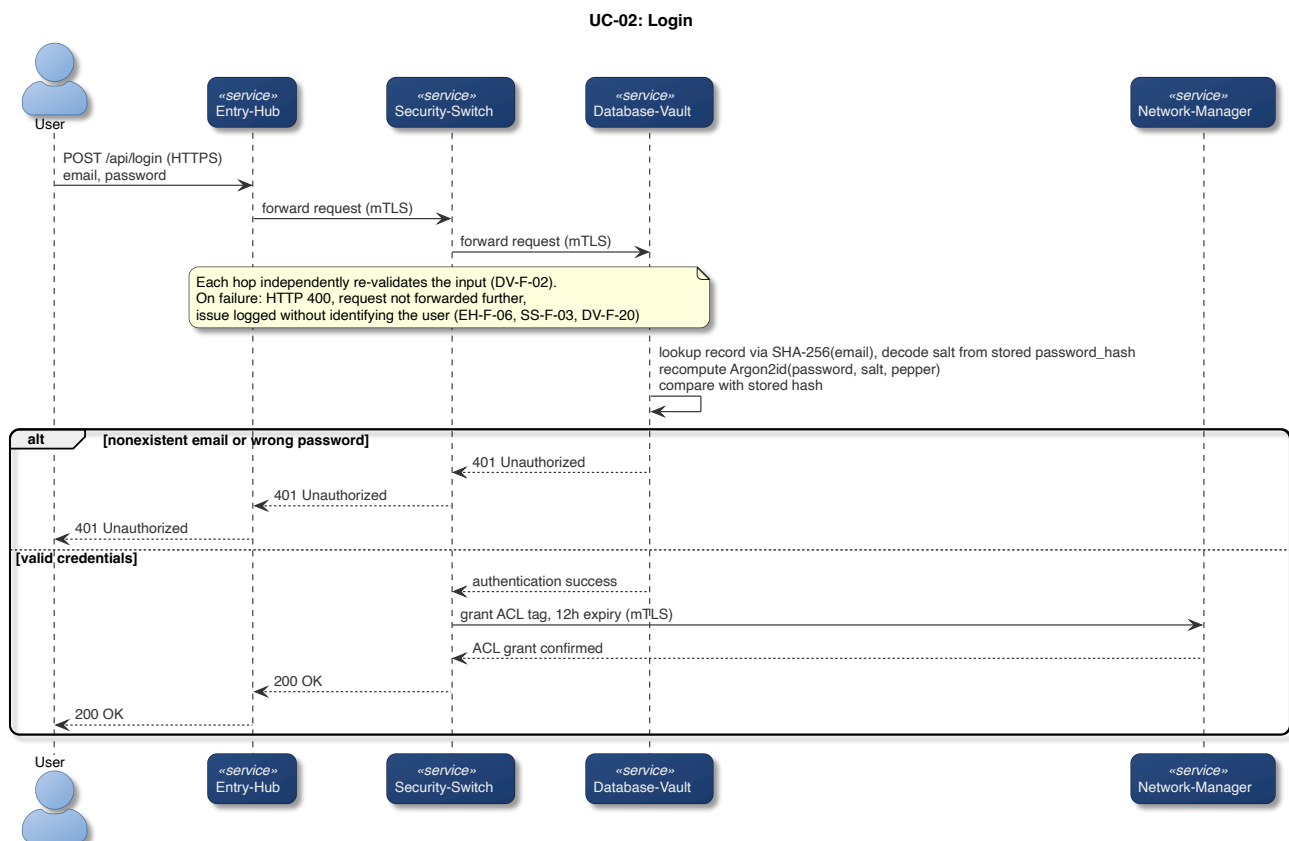


Figura 5.2: Sequenza di autenticazione (UC-02).

5.5 Orchestrazione post-autenticazione

Autenticare un utente non gli dà comunque ancora accesso a niente. L'accesso, in questo sistema, è una proprietà della rete. Security-Switch richiede a Network-Manager la modifica delle regole di rete in due momenti diversi.

Dopo una registrazione riuscita chiede la creazione dell'identità dell'utente sulla rete mesh e di una pre-auth key, che risale la catena fino al client dentro la risposta 201 e che permette al suo dispositivo di entrare nella rete privata.

Dopo un login riuscito chiede invece un accesso temporaneo di 12 ore verso Storage-Service.

Entrambi i passaggi concedono però raggiungibilità, non lettura: la registrazione mette il dispositivo nella rete, il login gli permette di raggiungere Storage-Service, ma ad aprire i backup è la chiave privata SSH, che non lascia mai il dispositivo dell'utente. Conoscere le credenziali dell'utente non è pertanto condizione sufficiente ad accedere ai suoi backup.

Come Network-Manager crei l'identità mesh, applichi il tag ACL e ne gestisca la scadenza è oggetto del Capitolo 7.

6

Storage e backup dei file

6.1 Modello di accesso SFTP-only

Storage-Service conserva i backup degli utenti ed è l'unico componente con cui il client parli direttamente dopo l'autenticazione. Una volta autenticato, infatti, l'utente diventa in grado di raggiungerlo sulla rete privata ed è quindi pronto per gestire i suoi backup.

Prima di trasmetterli però, il client dell'utente, tramite Restic [9], deduplica, comprime e cifra i backup dell'utente. Storage-Service riceve quindi una sequenza di blocchi cifrati che non è in grado di decifrare. Il client, all'inizializzazione della repository Restic, prende 32 byte da `crypto/rand`, li codifica in base64 e li usa come password del repository [43]. Da quella password Restic ricava la chiave che apre il file in cui custodisce la chiave di cifratura vera, generata a caso alla creazione del repository [44]: è quest'ultima a proteggere i blocchi.

Questo approccio è stato pensato per togliere la possibilità all'utente di scegliere una password debole. Il client salva la password del repository sul dispositivo dell'utente, con gli stessi permessi della chiave privata SSH. La sua segretezza, come la responsabilità di copiare le chiavi in un luogo sicuro, è lasciata all'utente. Nel caso in cui venisse persa anche una sola delle credenziali, l'utente non potrebbe in alcun modo accedere al contenuto dei suoi backup.

Il fatto che i dati degli utenti siano cifrati, però, non significa che debbano poter essere raggiunti da chiunque. L'unico modo per raggiungere i blocchi è infatti una sessione SFTP autenticata, all'interno della rete privata mesh. Su quella sessione, l'unica operazione possibile è trasferire file, e l'unica credenziale accettata è la chiave SSH [45].

Restano fuori gli altri modi di autenticarsi e gli altri usi che SSH consente di norma: eseguire un comando, aprire una shell, o servirsi della connessione come tramite per raggiungere altre macchine.

Inoltre, solo gli account creati dal servizio possono presentarsi, perché il server accetta esclusivamente nomi utente della forma `user*` e l'accesso root è esplicitamente disabilitato.

La lista predefinita di algoritmi crittografici di OpenSSH [45] è volutamente ampia per garantire compatibilità con client che non supportano gli algoritmi moderni e più robusti, ma qui quella compatibilità non serve, e rischia solo di indebolire il sistema, pertanto, anche gli algoritmi crittografici sono elencati a mano [46].

6.2 Isolamento e chroot dinamico

L'isolamento tra gli utenti si regge su un meccanismo del sistema operativo chiamato chroot, che confina un processo dentro una sottodirectory: da quel momento quella directory è, per lui, la radice del filesystem, e al di fuori di essa non esiste più alcun percorso che il processo possa nominare.

Alla registrazione, Storage-Service prepara per il nuovo utente l'account POSIX che abiterà quello spazio. L'account non ha una home directory tradizionale e la sua shell è `/usr/sbin/nologin`, un programma che si limita a rifiutare l'accesso.

Per ogni utente POSIX il cui nome corrisponde a `user*` vengono applicate anche le regole specificate nel Codice 6.1.

```
Match User user*
    ChrootDirectory /storage/%u # confina la sessione nel sottoalbero dell'utente
    ForceCommand internal-sftp # impone SFTP al posto del comando richiesto dal client
```

Codice 6.1: Le due direttive per utente, da `sshd_config` [46], con commenti aggiunti.

Accanto all'account nasce la struttura di directory illustrata nella Figura 4.3. La radice del confinamento appartiene a root, mentre la sottodirectory `data` appartiene all'utente ed è l'unico posto in cui possa scrivere. La proprietà da parte di root è un requisito del server SSH, che rifiuta di confinare una sessione in una directory scrivibile da chiunque altro.

Il risultato è che un utente collegato vede come radice del proprio filesystem la directory che gli appartiene, e le directory degli altri non sono raggiungibili, né elencabili in alcun modo.

Di norma un server SSH cerca le chiavi che autorizzano una connessione in un file dentro la home dell'utente, dove sono elencate quelle accettate per quell'account. Qui quel file non viene consultato. Al suo posto, a ogni tentativo di connessione, il server esegue un binario dedicato passandogli il nome utente che si sta presentando. Il binario gira con un'identità di sistema che non ha altri compiti sulla macchina e chiede a Database-Vault la chiave pubblica corrente di quell'utente su un canale mTLS.

Qualunque cosa vada storta produce lo stesso esito: il binario termina senza stampare nulla [47]. Il server riceve un elenco vuoto di chiavi e nega la connessione, secondo il principio fail-secure [28]. Non esiste alcun altro luogo al di fuori di Database-Vault in cui siano salvate le chiavi pubbliche SSH, quindi un suo malfunzionamento impedisce agli utenti di avviare le sessioni SFTP.

6.3 Backup e restore

Il backup avviene per intero sul dispositivo dell'utente. Il client invoca Restic indicando come repository la sottodirectory scrivibile dentro il proprio spazio confinato, raggiunta via SFTP, e gli impone di autenticarsi con la chiave privata generata alla registrazione [48], che non ha mai lasciato la macchina e che nessun componente del sistema ha mai visto.

Il percorso indicato al repository è già relativo alla radice del confinamento.

Il client risolve Storage-Service tramite MagicDNS sulla rete mesh, e la risoluzione riesce solo finché è attivo il permesso temporaneo concesso dall'ultimo login: scaduto quello, il nodo dell'utente non vede

più il servizio e la connessione non parte nemmeno. Il Capitolo 7 descrive come quel permesso venga concesso e revocato.

La Figura 6.1 riassume la sequenza, dalla connessione SFTP fino ai dati scritti.

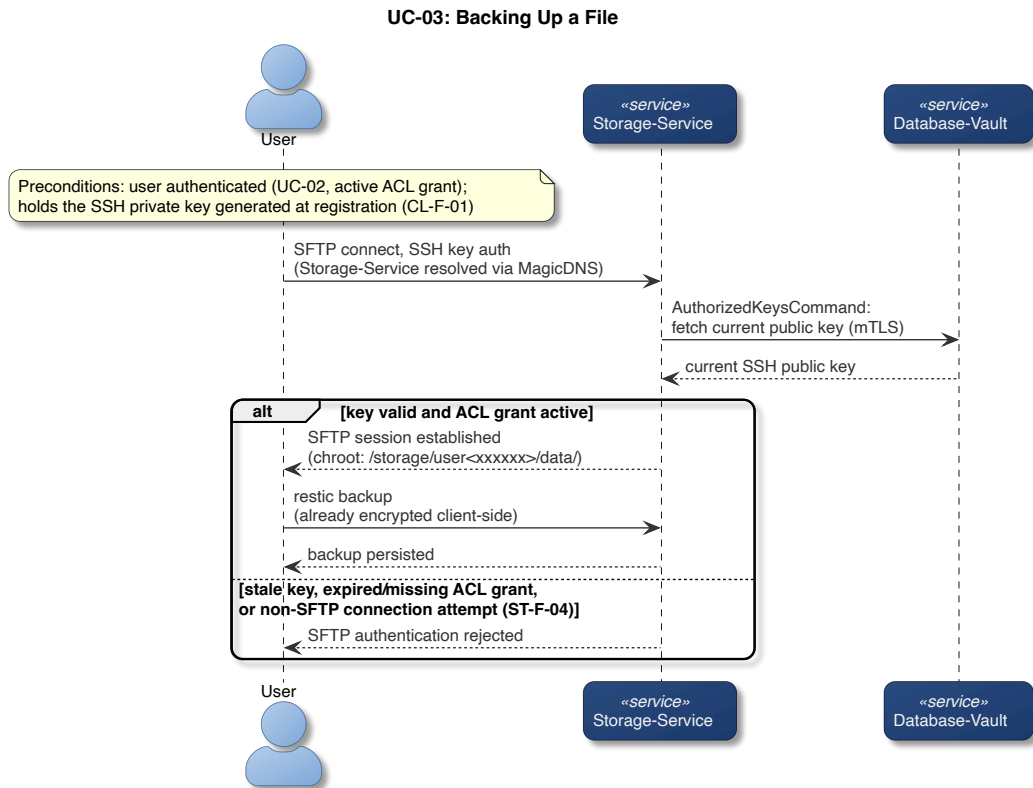


Figura 6.1: Sequenza di backup di un file (UC-03).

Il restore percorre la stessa strada in senso opposto, l'unica differenza è il comando impartito a Restic, che scarica i blocchi richiesti e li decifra sul dispositivo dell'utente.

Rete e coordinamento della mesh

7.1 Politiche di rete e confini di reachability

Le trust zones della Figura 4.4 descrivono chi può contattare chi. I confini di reachability che ne derivano sono resi effettivi da una politica di rete che Network-Manager pubblica su Headscale.

Ogni nodo della rete mesh, sia esso un servizio o il dispositivo di un utente, porta una o più etichette chiamate tag. Le regole di raggiungibilità sono scritte sui tag anziché su indirizzi IP, in modo da sopravvivere al cambiamento di indirizzo di un servizio, dovuto ad esempio al suo riavvio su un nodo diverso.

Network-Manager raccoglie tutte le regole in un unico documento e lo consegna a Headscale una sola volta, all'avvio [49].

Headscale a sua volta lo traduce in un filtro dei pacchetti: a ogni nodo che entra nella rete comunica solo i peer che le regole gli permettono di contattare, insieme ai propri aggiornamenti successivi. In questo modo, un servizio che da regolamento non deve contattarne un'altro, non è nemmeno in grado di vederlo nella rete.

Le regole, inoltre, nominano soltanto chi apre la connessione, quindi le risposte viaggiano senza una regola nel verso opposto. In questo modo, ad esempio, Security-Switch può contattare Database-Vault, e quest'ultimo può rispondere alla richiesta, ma non è in grado di avviare una comunicazione con lui.

7.2 Ciclo di vita dei grant di accesso

Le due regole che riguardano l'utente sono riportate nel Codice 7.1. Il tag `TagStorageAccess` è temporaneo e viene assegnato al login; `TagMeshMember` è permanente e viene assegnato quando il dispositivo entra nella rete.

```

{ // solo un utente autenticato, e Database-Vault, raggiungono Storage-Service
  Action: "accept",
  Src: []string{TagStorageAccess, TagDatabaseVault},
  Dst: []string{dstAny(TagStorageService)},
},
{ // ogni utente registrato raggiunge Entry-Hub
  Action: "accept",
  Src: []string{TagMeshMember},
  Dst: []string{dstAny(TagEntryHub)},
},

```

Codice 7.1: Le due regole che delimitano le zone dell'utente, da `policy.go` [49], con commenti aggiunti.

Alla registrazione, Network-Manager crea per il nuovo utente un'identità su Headscale e una pre-auth key, cioè una credenziale monouso con cui il dispositivo si presenta al coordinatore per entrare nella rete [50]. La chiave vale quindici minuti, e risale la catena dei servizi fino all'utente dentro la risposta alla registrazione.

La chiave nasce già associata al tag di sola appartenenza. Il dispositivo che la usa entra quindi nella rete e ottiene la raggiungibilità verso l'interfaccia di rete privata di Entry-Hub.

Al login Network-Manager cerca il nodo dell'utente e gli aggiunge il secondo tag, poi annota la scadenza a dodici ore da quel momento [51].

La ricerca del nodo dell'utente richiede però che Network-Manager tenga un piccolo database SQLite contenente l'identificativo numerico della pre-auth key legata all'hash dell'email dell'utente, con lo schema riportato nel Codice 7.2.

```

CREATE TABLE IF NOT EXISTS mesh_users (
  email_hash TEXT PRIMARY KEY,
  pre_auth_key_id INTEGER NOT NULL
);

```

Codice 7.2: Schema della tabella di corrispondenza email-pre-auth-key, da `meshusers.go` [52].

Anche le scadenze dei tag sono persistite sul database, per evitare che un riavvio del servizio trasformi un permesso di dodici ore in un permesso perpetuo. Lo schema della tabella che le contiene è riportato nel Codice 7.3.

```

CREATE TABLE IF NOT EXISTS grants (
  email_hash TEXT PRIMARY KEY,
  node_id INTEGER NOT NULL,
  tag TEXT NOT NULL,
  expires_at INTEGER NOT NULL
);

```

Codice 7.3: Schema della tabella dei grant di accesso, da `store.go` [53].

Un processo periodico, ogni cinque minuti, cerca le scadenze passate e revoca i grant scaduti [54]. La riga viene cancellata dal database prima di chiamare Headscale per togliere il tag, e la cancellazione è condizionata alla scadenza esatta appena letta. Se nel frattempo un login ha rinnovato il grant, la

scadenza in tabella non coincide più, la cancellazione non tocca nessuna riga, e lo sweep si ferma senza contattare Headscale. Chiamare prima Headscale e cancellare poi avrebbe invece rischiato di cancellare anche la riga appena riscritta dal login concorrente, togliendo all'utente un permesso appena concesso.

La Figura 7.1 riassume gli stati che il nodo di un utente attraversa.

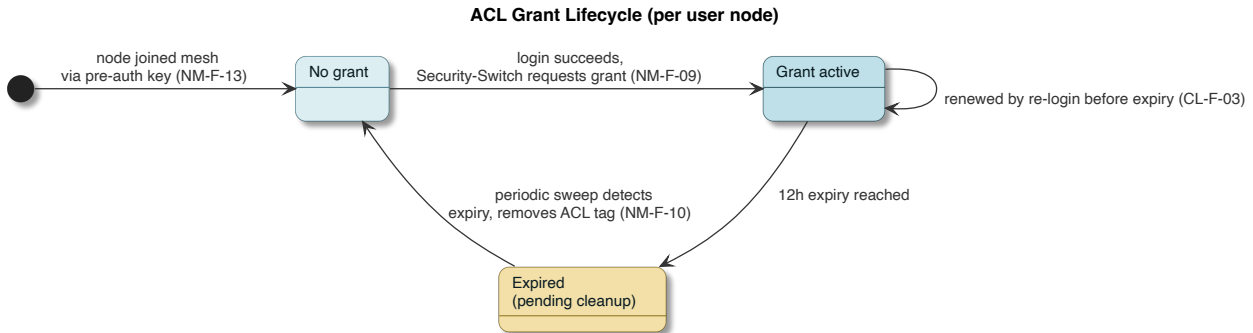


Figura 7.1: Ciclo di vita del permesso di accesso associato al nodo di un utente.

7.3 Il paradosso del coordinatore: Headscale come servizio a sé

La documentazione di Headscale dichiara che non è consigliato eseguire Headscale su una macchina che appartiene alla rete coordinata, perché può creare problemi alla risoluzione dei nomi tramite MagicDNS [55].

Il coordinatore deve perciò restare estraneo alla rete che coordina, e da qui discendono tre conseguenze.

La prima è che non può condividere una macchina virtuale o un container con nessun altro componente, perché tutti gli altri sono membri della mesh.

La seconda è che il traffico con cui Network-Manager crea le chiavi e assegna i tag non può essere protetto dalla collocazione in rete, come invece accade per ogni altra chiamata interna. È l'unica chiamata tra componenti del sistema che attraversa la rete pubblica, e l'unica la cui protezione dipende interamente da mTLS.

La terza è che il coordinatore deve essere raggiungibile pubblicamente, perché un dispositivo che non è ancora entrato nella rete deve poterlo contattare la prima volta per poter entrare nella rete.

Headscale ha però una limitazione rilevante per RAM-USB: non offre alcun modo di separare il protocollo di coordinamento, che deve essere raggiunto da tutti, dall'API amministrativa, che deve essere raggiunta solo da Network-Manager. La separazione è stata affidata a un reverse proxy, che distingue le due superfici in base al percorso della richiesta [56]. Le richieste di coordinamento passano senza alcun controllo, mentre quelle amministrative sono accettate solo se accompagnate da un certificato emesso dalla Certificate-Authority privata, a cui si aggiunge anche una chiave API che Headscale richiede per conto proprio. Il traffico amministrativo è difeso da due credenziali indipendenti perché è una delle superfici più sensibili.

Il certificato presentato dal proxy è invece emesso da una CA pubblica, per la stessa ragione per cui lo è quello di Entry-Hub.

7.4 CG-NAT e la necessità di un endpoint pubblico dedicato

Visto che gli indirizzi IPv4 sono ormai terminati, gli Internet Service Providers assegnano ai router domestici dei clienti un IPv4 condiviso tra più clienti nel range `100.64.0.0/10` rendendo impossibile ai clienti esporre un proprio server su internet. Questo tipo di natting prende il nome di Carrier-Grade NAT [57]. Visto che RAM-USB è progettato per essere hostato anche da utenti consumer, che potrebbero quindi non possedere un IPv4 pubblico o un IPv6, Headscale ed Entry-Hub risiedono su un Virtual Private Server, separato dal resto del sistema. Il Capitolo 10 descrive la collocazione effettiva di ciascun componente.

8

Osservabilità e metriche

8.1 Pubblicazione delle metriche

Le metriche operative possono tipicamente raggiungere chi le osserva in due modi: o è qualcun altro a interrogare periodicamente ogni servizio (*pull*), oppure è ogni servizio a inviarle di propria iniziativa (*push*). RAM-USB adotta il secondo: ogni servizio pubblica le proprie metriche una volta al minuto, sul proprio topic MQTT dedicato verso il broker Mosquitto [58]. La scelta deriva dal fatto che un modello *pull* richiederebbe che Metrics-Collector apra una connessione verso ciascuno dei sei servizi, ampliando le regole di reachability della mesh discusse nel Capitolo 7. Pubblicando di propria iniziativa, sono invece i servizi ad aprire la connessione verso il broker. Da lì, Metrics-Collector legge le metriche e le persiste in un database.

Le metriche operative non contengono dati degli utenti, ma i metadati del sistema vanno protetti con la stessa cura, per questo le comunicazioni avvengono all'interno della rete privata e protette dal mTLS, certificato dalla CA interna.

Ogni pubblicazione porta come corpo un payload di sei campi, riportato nel Codice 8.1. Le metriche nel payload fanno riferimento al minuto in cui sono state inviate.

```
type Payload struct {  
    Service string `json:"service"`  
    Timestamp string `json:"timestamp"`  
    RequestCount int64 `json:"request_count"`  
    ErrorCount int64 `json:"error_count"`  
    AverageResponseTimeMs float64 `json:"average_response_time_ms"`  
    ActiveConnections int64 `json:"active_connections"`  
}
```

Codice 8.1: Struttura del payload delle metriche, da `payload.go` [59].

Le pubblicazioni usano tutte *QoS 1*, (at least once) nella classificazione MQTT [60]: il broker garantisce che il messaggio arrivi almeno una volta, al costo di un possibile duplicato. *QoS 0*, (at most once), è escluso perché RAM-USB non può permettersi di perdere metriche in silenzio; *QoS 2*, (exactly once), è escluso perché la garanzia in più che offre (l'assenza di duplicati) qui non serve visto che un duplicato si traduce solo in due punti sovrapposti nello stesso istante, invisibili sul grafico.

8.2 Ricezione e controllo di coerenza

Il messaggio pubblicato deve arrivare a un solo destinatario, e a nessun altro. Questa proprietà è garantita dall'ACL di Mosquitto che garantisce che ogni servizio possa solo scrivere sul proprio topic, mentre Metrics-Collector può leggere dai topic `metrics/#` senza potervi scrivere.

Quell'ACL, però, non impedisce che un servizio compromesso, pur restando entro il proprio permesso di scrittura legittimo, pubblichi un payload il cui campo `service` dichiarato sia un nome diverso dal proprio. Metrics-Collector rivalida quindi ogni messaggio in modo indipendente, derivando il servizio atteso dal topic di arrivo e confrontandolo con quello dichiarato nel payload. Su qualunque discrepanza scarta il messaggio e lo registra nei log.

8.3 Persistenza delle serie temporali

Metrics-Collector persiste le metriche in TimescaleDB [30], un'estensione di PostgreSQL pensata per un carico fatto quasi solo di scritture ordinate nel tempo.

La tabella `metrics` non ha una chiave primaria dato che una riga di metriche non ha un'identità naturale, inoltre, TimescaleDB [30] non ne richiede una. È organizzata in chunk per intervalli di tempo sulla colonna `time`: le due politiche in background del Codice 8.2 agiscono chunk per chunk, in base alla loro età. La prima elimina ogni chunk più vecchio di trenta giorni; la seconda, dopo sette giorni, sposta i chunk in un formato compresso a colonne, raggruppato per servizio e ordinato per tempo decrescente.

```
SELECT create_hypertable('metrics', by_range('time'));

SELECT add_retention_policy('metrics', drop_after => INTERVAL '30 days');

ALTER TABLE metrics SET (
    timescaledb.enable_columnstore,
    timescaledb.segmentby = 'service',
    timescaledb.orderby = 'time DESC'
);
CALL add_columnstore_policy('metrics', after => INTERVAL '7 days');
```

Codice 8.2: Politiche automatiche di ritenzione e compressione della tabella `metrics`, da `000001_create_metrics_table.up.sql` [61].

8.4 Visualizzazione delle metriche

Un amministratore consulta lo stato di salute del sistema, senza che quella consultazione tocchi mai i dati degli utenti visto che per costruzione, le metriche non ne contengono. Grafana [31] legge da TimescaleDB direttamente.

Per contattare TimescaleDB, oltre alla raggiungibilità sulla rete mesh e al certificato mTLS di Grafana, è richiesta anche una password, sullo stesso principio di difesa in profondità già visto per l'API amministrativa di Headscale nel Capitolo 7.

La dashboard mostra tre pannelli, ciascuno una query diretta sulla tabella `metrics`: tempo medio di risposta, throughput e connessioni attive.

La Figura 8.1 riassume l'intera pipeline appena descritta, dalla pubblicazione sul topic MQTT alla dashboard.

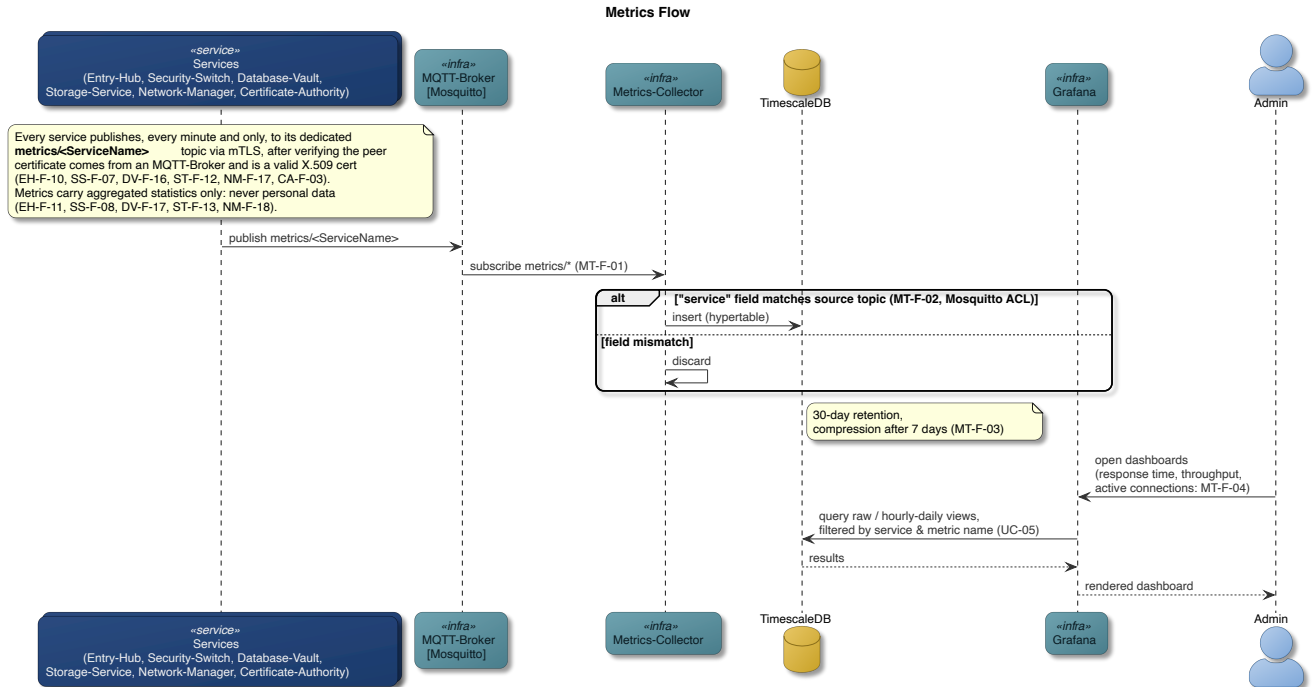


Figura 8.1: Pipeline di osservabilità, dalla pubblicazione delle metriche alla dashboard di Grafana.

Testing e validazione

9.1 Il modello V e i quattro livelli di test

Per il testing di RAM-USB è stato adottato il modello a V, un'estensione del modello a cascata in cui ad ogni fase di specifica viene affiancata una fase di verifica corrispondente, scritta di pari passo con la specifica.

Questo approccio è stato adottato in quanto si è scelto di trattare RAM-USB come un sistema safety-critical, in cui un difetto in produzione ha conseguenze serie sugli utilizzatori del sistema. Era pertanto fondamentale che un errore nella specifica venisse rilevato molto prima dell'implementazione e che il testing venisse applicato su vari livelli di specifica.

La Tabella 9.1 riporta la corrispondenza tra la categoria del requisito e il livello di test che lo verifica, con la tecnica usata a quel livello.

Tabella 9.1: Livello di verifica associato a ciascun livello di specifica.

Livello di specifica	Livello di verifica
Requisito di componente	Test di unità con il pacchetto <code>testing</code> di Go
Caso d'uso	Test di integrazione, bottom up con driver e stub
Requisito non funzionale	Test di sistema, sull'intero stack
Requisito utente	Lista di accettazione, percorsa a mano sul caso d'uso corrispondente

9.2 Strategia di integrazione bottom-up

L'integrazione parte dal componente più interno e procede verso l'esterno, nell'ordine Database-Vault, Storage-Service, Security-Switch, Network-Manager, Entry-Hub.

La scelta di partire da Database-Vault è dettata dal fatto che questo è contemporaneamente il componente con meno dipendenze uscenti e quello su cui si concentra il maggior numero di requisiti funzionali. Costruirlo per primo, sostituendo con stub tutto ciò che gli sta sopra, fa sì che il componente più sensibile sia anche quello verificato più a lungo.

9.3 TDD: criteri di ingresso e uscita

Il criterio di ingresso è che per ogni requisito esista un test, e che l'eventuale fallimento di quel test avvenga prima dell'implementazione del requisito da testare.

Un esempio: il requisito DV-F-15 richiede che un'email inesistente e una password sbagliata producano lo stesso esito di login, indistinguibile dall'esterno.

Il test che lo verifica, `TestLogin.NonexistentEmailAndWrongPassword.AreIndistinguishable` [62], chiama `Login` con le due credenziali diverse e pretende che l'esito coincida esattamente.

Un test mai visto fallire, magari scritto dopo il codice e passato al primo colpo, non garantisce di verificare nulla: potrebbe confrontare due valori uguali per costruzione, o non chiamare affatto la funzione che dovrebbe testare. Per questo motivo, il test è scritto prima che il body della funzione `Login` esista davvero, e fallisce fornendo la prova che il confronto nel test sa riconoscere un esito sbagliato e ritorna l'output atteso: `Outcome differs: nonexistent email = ..., wrong password = ....`

Solo dopo il fallimento del test ha senso scrivere il body completo di `Login`, e solo allora il passaggio da test fallito a test passato diventa un'informazione utile.

Scrivere i test prima ha anche un'altro vantaggio: aiuta a disambiguare e a chiarire i requisiti. Se per un requisito non si riesce a scrivere un test, infatti, significa che il requisito è poco chiaro o ambiguo, di conseguenza, viene riformulato meglio.

I criteri di uscita sono tre, e dipendono dal codice da testare. Il primo riguarda solo la libreria di validazione, che richiede test più approfonditi perché considerata particolarmente sensibile.

- nella libreria di validazione condivisa da Entry-Hub, Security-Switch e Database-Vault (sezione 5.1), ogni ramo di ogni `if` o `switch` che decide se accettare o rifiutare un campo deve essere attraversato da almeno un caso di test;
- nessun test già passato sulla catena di componenti già integrata può fallire;
- una modifica non riesegue solo il test scritto apposta per lei, ma l'intera suite di test.

9.4 Copertura e lacune del piano di test

`go test -cover` misura, per ogni pacchetto della codebase, la percentuale di istruzioni eseguite almeno una volta: `pkg/validation` ad esempio, arriva al 98%, perché lì vive la logica applicativa, mentre i pacchetti `cmd` che avviano ciascun servizio restano al 10-20%, perché verificarli è compito dei test di integrazione e di sistema.

Il criterio di uscita citato sopra, per `pkg/validation`, è una misura più severa e si aggiunge a questa: non basta eseguire quasi tutte le istruzioni, ogni singolo ramo di ogni `if` o `switch` deve essere stato preso da almeno un test.

Il piano di test però, non è esente da limitazioni: Non esiste alcun test previsto né per la protezione dei backup dalla manomissione, né per la rotazione della master key, attualmente fuori perimetro, ma fondamentali in un progetto di questo tipo. I test di carico e di stress, inoltre, sono previsti, ma non ancora progettati. Non si hanno quindi benchmark affidabili del comportamento del sistema. Ogni test end to end sull'host ha riguardato un solo utente con un solo piccolo file nel backup, quindi né la concorrenza né l'isolamento reciproco fra due utenti sono stati messi alla prova.

9.5 Verifica automatica pre-commit

Oltre ai criteri di ingresso e di uscita, una catena di verifica automatica viene eseguita prima di ogni commit [2]. `golangci-lint` [63] è un esecutore che aggrega più linter in un solo passaggio; quello configurato per RAM-USB ne abilita quindici in tutto.

Tra questi, `gosec` [64] cerca pattern di sicurezza noti nel codice, ed è presente fin dalle prime fasi di implementazione.

Una seconda estensione ha introdotto quattro linter mirati a risorse mai rilasciate e a errori gestiti male:

- `bodyclose` [65] segnala una risposta HTTP il cui corpo non viene mai chiuso, che porta quindi ad una perdita di risorse;
- `sqlclosecheck` [66] fa lo stesso per righe e statement SQL rimasti aperti;
- `contextcheck` [67] individua una funzione che non propaga il `context.Context` ricevuto dal chiamante;
- `errorlint` [68] verifica che gli errori vengano confrontati nel modo corretto.

A questi si affianca il rilevatore di race-condition di Go, invocato con `go test -race`.

Completano la catena `govulncheck` [69], che confronta le dipendenze con un database di vulnerabilità note ma segnala un problema solo se il codice raggiunge davvero la funzione vulnerabile, e due scanner pensati per sovrapporsi tra loro: `gitleaks` [70], che cerca segreti già committati nella storia del repository, e `trivy` [71], invocato in modalità di scansione del filesystem (`trivy fs .`), che verifica sia le dipendenze sia i segreti in modo indipendente dai due strumenti precedenti.

La ridondanza degli ultimi due è l'applicazione ai test del concetto di difesa in profondità.

9.6 Risultati: verifiche completate

La campagna più significativa è stata eseguita su hardware reale, un host Proxmox con i componenti distribuiti secondo la collocazione descritta nel Capitolo 10.

I quattro casi d'uso sono passati: registrazione e login sono riusciti, il backup si è concluso e il ripristino ha restituito un file identico all'originale.

La Tabella 9.2 riporta le proprietà controllate sul sistema in esecuzione.

Tabella 9.2: Proprietà verificate sul sistema in esecuzione.

Proprietà	Riscontro
Parametri di hashing	La stringa salvata nel record riporta <code>argon2id</code> , versione 19, memoria 47104 KiB, due passaggi, parallelismo uno
Isolamento su filesystem	Radice del confinamento di proprietà di root con permessi 711, sottodirectory dell'utente 700
Identità sulla mesh	Ogni nodo risulta di proprietà dell'utente sintetico <code>tagged-devices</code> , mai dell'account creato per la persona
Durata del permesso	Scadenza registrata a dodici ore esatte dall'istante del login
Confini di raggiungibilità	Nessun nodo elenca fra i propri peer un componente che la politica gli vieta di contattare
Zero-knowledge	Nessuna occorrenza del testo in chiaro fra i blocchi salvati, e la configurazione del repository era essa stessa cifrata
Sopravvivenza al riavvio	Un servizio fermato e riavviato torna in servizio riusando l'identità su disco, senza ripresentare il token monouso già consumato

Durante l'esecuzione del piano di test completo, sono emersi diversi difetti non intercettati dai test di unità che li precedevano [5].

Due riguardavano il client e impedivano del tutto il backup. Il primo era la porta SSH: il client si collegava su quella predefinita, mentre il server ascolta su un'altra. Il secondo era l'assenza di qualunque politica sulla chiave host: `ssh`, invocato da Restic senza terminale, non aveva un `known_hosts` da controllare né modo di chiedere conferma, e rifiutava la connessione anziché chiederla, quindi il primo backup su ogni macchina nuova falliva sempre. Un terzo riguardava la configurazione, perché il client dispone di un solo indirizzo per un servizio che ne espone due su listener diversi. Un quarto era una stringa d'uso che documentava una sintassi che il parsing degli argomenti non può accettare.

Un quinto, più grave: l'identità mTLS non veniva conservata fra un avvio e il successivo, quindi nessun servizio sopravviveva al riavvio del proprio container, perché ritentava il bootstrap con un token monouso ormai speso.

10

Deployment

10.1 Containerizzazione e supervisione s6-overlay

Un container Docker nasce pensato per un solo processo. Alcuni servizi di RAM-USB però, devono far girare più processi indipendenti nello stesso container, e necessitano quindi di un supervisore che li avvii nell'ordine giusto e li riavvii quando cadono.

RAM-USB usa `s6-overlay` [72] per questo scopo. Ogni processo supervisionato è una directory sotto `s6-rc.d/`, dichiarata di tipo `longrun`, con le proprie dipendenze elencate come file dentro una sottodirectory `dependencies.d/`: la dipendenza non è configurazione da scrivere, ma la semplice esistenza di un file il cui nome è quello del processo atteso. L'ordine di avvio nasce quindi da un grafo dichiarato, non da uno script che indovina quanto aspettare.

Storage-Service è il caso più completo: quattro processi indipendenti convivono nello stesso container [73, 74]. `tailscaled`, il client mesh reale, si unisce alla rete solo dopo aver instaurato fiducia nella CA della mesh.

Il server `sshd` irrigidito (Capitolo 6) dipende da tre condizioni insieme: le proprie host key generate, la mesh già raggiunta, e la prima scrittura riuscita di `identity-provisioner`, il quarto processo, che mantiene aggiornata su disco l'identità mTLS con cui `AuthorizedKeysCommand` interroga Database-Vault. Il binario Go dello stesso Storage-Service, invece, dipende solo dalle prime due: non chiama mai quell'identità, quindi non deve attenderla.

Non tutti i servizi hanno bisogno di questa complessità. Entry-Hub non supervisiona alcun processo secondario: la sua appartenenza alla mesh non vive nello stesso container, ma in un sidecar separato, `entry-hub-mesh`, che condivide il suo network namespace. Il paragrafo seguente spiega perché questa differenza non è un dettaglio implementativo, ma il criterio con cui è stata scelta l'immagine di base di ciascun servizio.

10.2 Perché non tutti i container sono distroless

Un'immagine distroless [21] non ha shell né gestore di pacchetti, riducendo così la maggior parte delle vulnerabilità critiche note nei container [22, 23]. Un supervisore come `s6-overlay`, però, è a sua volta un piccolo sistema di init: ha bisogno di una shell per i propri script di avvio, esattamente ciò che

un'immagine distroless non fornisce. Qualunque servizio debba far girare più di un solo binario Go non può quindi essere distroless.

Di conseguenza, l'unico servizio davvero distroless è Entry-Hub: l'unico la cui appartenenza alla mesh è esternalizzata in un container separato, senza bisogno di eseguire nient'altro accanto al proprio binario [75].

Gli altri cinque, Security-Switch, Storage-Service, Network-Manager, Database-Vault e Metrics-Collector, hanno ciascuno una ragione OS-level specifica che esclude distroless: un `tailscaled` reale, che ha bisogno di una vera interfaccia di rete a livello di kernel; `sshd` e la creazione di utenti POSIX, nel caso di Storage-Service. Tutti usano quindi `debian:bookworm-slim` come immagine di base [76].

Tabella 10.1: Immagine di base per servizio e motivo del bisogno OS-level.

Servizio	Immagine di base	Bisogno OS-level
Entry-Hub	distroless	nessuno: mesh esternalizzata in un sidecar
Security-Switch	debian:bookworm-slim	<code>tailscaled</code> reale
Storage-Service	debian:bookworm-slim	<code>tailscaled</code> reale, <code>sshd</code> , utenti POSIX
Network-Manager	debian:bookworm-slim	<code>tailscaled</code> reale
Database-Vault	debian:bookworm-slim	<code>tailscaled</code> reale
Metrics-Collector	debian:bookworm-slim	<code>tailscaled</code> reale

Questa asimmetria, cinque servizi su sei fuori dal caso di default, è l'opposto di come la SRS [1] inquadra la policy: la descrive come regola generale con Storage-Service come sola eccezione. Il criterio che decide, il bisogno OS-level, resta corretto; è la sua conta a essere cambiata via via che i servizi hanno preso un client mesh reale invece che una libreria in-process.

10.3 Docker locale vs Proxmox VE: due livelli di isolamento

Sarebbe naturale pensare a Docker e a Proxmox VE [77] come due alternative per la stessa esigenza di isolamento. Non lo sono: Docker garantisce portabilità e isolamento per-servizio, identici in sviluppo e in produzione, mentre Proxmox aggiunge sotto ciascun container Docker un confine di isolamento più forte, quello di una macchina virtuale o di un container di sistema. Ogni servizio gira quindi dentro il proprio container Docker, così come in sviluppo, ma quel container Docker gira a sua volta dentro la sua VM o il suo container Proxmox assegnato in produzione.

RNF-ORG-04 assegna Storage-Service, Database-Vault e Network-Manager a guest KVM, e i restanti servizi ospitati su Proxmox, Security-Switch, Certificate-Authority, Grafana, Metrics-Collector e Mosquitto, a container LXC [78, 79]. Entry-Hub e Headscale restano fuori da Proxmox del tutto, su un VPS pubblico dedicato ciascuno: il Capitolo 7 spiega già perché.

La scelta tra KVM e LXC dipende dallo stesso bisogno di interfaccia di rete reale già visto nella sezione 10.2. `tailscaled` ha bisogno di un'interfaccia di rete a livello di kernel: in un container LXC non privilegiato serve un passthrough esplicito di `/dev/net/tun`, mentre una VM KVM la fornisce senza configurazione aggiuntiva [78].

Storage-Service ha inoltre bisogno di eseguire `setuid` e `chroot` per isolare gli utenti POSIX (sezione 6.2): farlo dentro un container LXC, già esso stesso un namespace, aggiungerebbe un secondo

livello di mappatura degli UID sopra il primo. Una VM intera evita di sovrapporre due meccanismi di isolamento che non compongono in modo pulito [73].

10.4 Topologia distribuita

Il repository non contiene un unico file Compose che orchestri l'intero sistema, ma undici file separati, uno per componente [80]. La scelta rispecchia deliberatamente la distribuzione reale: in produzione questi processi non condividono affatto lo stesso host, quindi un unico file fonderebbe insieme una topologia che nella realtà è già divisa su VM, container LXC e VPS distinti, e andrebbe smontato di nuovo al momento del deployment.

Ogni file Compose ha una controparte scritta a mano sotto `deployments/proxmox/` o `deployments/vps/`, che traduce le variabili d'ambiente e il cablaggio di quel servizio in note per il guest Proxmox o VPS corrispondente [73, 81]. Questa traduzione non reinventa nulla: riprende gli stessi commenti di documentazione già presenti nel codice e nei Dockerfile di ciascun servizio, applicando alla topologia distribuita la stessa disciplina di tracciabilità già descritta nel capitolo 3.

10.5 Alta disponibilità e resilienza: stato attuale e limiti

Il Capitolo 4 lascia aperta una domanda: il servizio resta disponibile agli utenti solo finché restano disponibili i sette componenti che partecipano al percorso di una richiesta. Rispondere richiede di distinguere tre piani diversi, invece di limitarsi a rimandare a Proxmox.

Il primo è ciò che esiste oggi: la supervisione di `s6-overlay` riavvia un processo caduto dentro un container (sezione 10.1), e il barrido periodico di Network-Manager (Capitolo 7) tiene pulito lo stato dei permessi scaduti. Nessuno dei due copre però la caduta della macchina che ospita il container: coprono il crash di un processo, non l'indisponibilità del nodo fisico o virtuale su cui gira.

Il secondo è ciò che è solo potenziale. Un cluster Proxmox iperconvergente offrirebbe live migration e failover automatico tra nodi fisici, ma richiede almeno tre nodi per il quorum e uno storage replicato come Ceph: la stessa nota della SRS che introduce questa possibilità [1] la mette al condizionale, "se un cluster iperconvergente sarà costruito in seguito", proprio perché nel deployment attuale non esiste. È quindi una direzione, non una mitigazione già in campo.

Il terzo vale anche una volta che il secondo fosse costruito: il failover copre la caduta di un nodo fisico, non un errore di configurazione né un processo che risponde in modo degradato pur restando in esecuzione, e costa comunque il riavvio della VM sull'altro nodo. Il risultato sarebbe quindi minuti di indisponibilità, non continuità di servizio.

A questi tre piani si aggiunge un limite già accettato per un motivo diverso dalla disponibilità del nodo fisico: RISK-04 della SRS [1]. Il nodo mesh di Network-Manager accetta il MagicDNS distribuito da Headscale per risolvere i propri hostname in uscita verso Certificate-Authority e verso il broker MQTT, ma quella stessa risoluzione dipende dalla policy ACL che Network-Manager pubblica su Headscale.

Se quella policy venisse pubblicata in uno stato non funzionante, Network-Manager potrebbe perdere la capacità di risolvere anche l'hostname di cui avrebbe bisogno per raggiungere l'API amministrativa di Headscale e correggerla. La scelta di accettare comunque il MagicDNS, invece di costruire un percorso

di recupero automatico, è stata presa il 26 luglio 2026 a favore della semplicità: in questo scenario di guasto, un operatore interviene manualmente.

Il Capitolo 11 riprende il costo complessivo di questi limiti nel bilancio delle scelte architetturali.

Analisi critica delle proprietà di sicurezza

11.1 Scenari di attacco e mitigazioni

I capitoli precedenti descrivono le difese di RAM-USB dal punto di vista di chi le costruisce. Questa sezione le rilegge dal punto di vista opposto, quello di chi tenta di romperle, senza ripetere i meccanismi già visti: solo dove la difesa regge, e dove non basta la disciplina del singolo sviluppatore a garantirla.

Due garanzie del sistema sono infatti imposte a livello di tipo, non di convenzione. Il pacchetto `pkg/errors` lega ogni costruttore a un codice HTTP fisso e a un messaggio pubblico altrettanto fisso: chi scrive un handler non ha modo di costruire una risposta 401 con un messaggio diverso da quello sanitizzato, perché il costruttore non accetta un messaggio pubblico come parametro.

```
func NewUnauthorized(internal error) *AppError {
    return &AppError{
        Status: http.StatusUnauthorized,
        Public: "authentication failed",
        Internal: internal,
    }
}
```

Codice 11.1: Il messaggio pubblico è fisso nel costruttore, non scelto dal chiamante, da `errors.go` [82].

Allo stesso modo, `pkg/logging` tipizza email e password con un tipo dedicato, `Redacted`, che implementa `slog.LogValuer`: qualunque valore incapsulato in quel tipo stampa sempre la stringa `REDACTED`, indipendentemente da cosa contenga davvero. La garanzia DV-F-03/RD-01, nessuna credenziale nei log, non dipende quindi dal fatto che chi scrive una nuova chiamata di log se ne ricordi, ma dal fatto che il compilatore non lascia altra scelta a chi maneggia un valore di quel tipo.

```
type Redacted string

func (r Redacted) LogValue() slog.Value {
    return slog.StringValue("REDACTED")
}
```

Codice 11.2: Il tipo che impedisce di loggare una credenziale in chiaro, da `logging.go` [83].

Queste due garanzie proteggono cosa esce da un componente onesto. Non proteggono, però, da un componente che si presenta con l'identità sbagliata o da una credenziale usata più volte.

La compromissione isolata di Entry-Hub, per esempio, non basta a impersonare Security-Switch verso Database-Vault, perché il certificato di Entry-Hub porta il campo `organization` sbagliato, e la mTLS lo rifiuta indipendentemente dal fatto che il certificato sia valido (sezione 4.4). Una pre-auth key rubata, allo stesso modo, non si può ripresentare una seconda volta, perché è pensata per un solo utilizzo (Capitolo 7): un componente compromesso resta confinato a ciò che quel componente, con la sua identità, può davvero raggiungere.

11.1.1 Enumerazione degli indirizzi email in registrazione

`POST /api/users` è un endpoint pubblico e non autenticato che risponde 201 se l'email inviata era libera e 409 se era già registrata, senza specificare quale campo, email o chiave SSH, abbia causato il conflitto. Chi invia però una chiave SSH generata al momento, quindi certamente unica, ha già escluso l'unica altra causa possibile: la risposta diventa così inequivocabile, e chiunque può testare un indirizzo alla volta, semplicemente osservando il codice HTTP restituito.

Questa è precisamente l'informazione che il login nasconde: DV-F-15 restituisce lo stesso 401 sia per un'email inesistente sia per una password sbagliata, con lo stesso log e con lo stesso tempo di risposta (sezione 5.1). Quella protezione, però, non è mai stata estesa alla registrazione.

L'effetto collaterale è più grave della sola fuga di informazione. EH-F-04 valida solo il formato dell'email, non il suo possesso: non esiste alcun link di conferma. Una sonda che ottiene 201 registra quindi davvero un account su quell'indirizzo, che resta per sempre non registrabile dal suo legittimo proprietario, dato che DV-F-12 risponderà 409 per sempre e non esiste alcun percorso di cancellazione.

Questo problema è tracciato come KI-106 in `docs/Known-Issues.md` [5], non tra i quattro rischi accettati della SRS: per ora resta solo osservato, non ancora promosso a rischio formale.

11.1.2 Accesso amministrativo alle credenziali

RNF-SEC-01 ammette che email e password attraversino, cifrate in transito, Entry-Hub, Security-Switch e Database-Vault per la validazione e l'hashing. Chi controlla uno di questi tre processi può leggerle nel punto in cui sono in chiaro in memoria: è il costo che un sistema di autenticazione classico paga rispetto a un protocollo zero-knowledge come SRP, ed è precisamente il compromesso che questo progetto ha scelto di esplorare invece di evitare, per studiare fin dove le sue garanzie possano essere spinte.

A riposo il quadro cambia. La password non è mai recuperabile, nemmeno da un amministratore con pieno accesso al database, perché Argon2id non si inverte (sottosezione 5.2.1). L'email lo è, ma solo da

chi possiede la master key. RU-06 vale quindi a riposo, non contro un amministratore malintenzionato che controlli uno dei tre processi lungo il percorso della richiesta.

11.1.3 Verifica offline di un indirizzo a partire da una copia del database

Le due sezioni precedenti riguardano un attaccante che interroga l'API pubblica. Questa riguarda un attaccante che possiede una copia rubata del database, e la scala di ciò che serve ai due è diversa. Per enumerare via 409 basta l'accesso di rete all'API, e nessun dato posseduto in anticipo. Per verificare un indirizzo candidato offline basta invece il dump, senza alcuna chiave: la chiave di ricerca dell'email è uno SHA-256 non tenuto sotto chiave, quindi ricalcolabile su qualunque candidato. Decifrare l'email vera e propria richiede infine il dump insieme alla master key.

Il limite di questo secondo scenario va dichiarato senza sopravvalutarlo: non si stanno invertendo gli hash, si stanno testando dei candidati. Un dizionario di indirizzi comuni, però, copre già molto, quindi dire che l'attaccante "non può ricavare gli indirizzi che non sospetta" è vero in senso stretto e debole in pratica.

L'esposizione, inoltre, è più ampia di quanto un dump da solo suggerisca. Lo stesso digest SHA-256 finisce anche su Headscale: il nome utente mesh di ogni account è la lettera u seguita dai primi 24 caratteri esadecimali dello stesso hash, mostrato da `headscale users list` accanto all'email in chiaro dello stesso account.

Chiunque abbia accesso in lettura a Headscale, un host deliberatamente pubblico e fuori dalla mesh privata (Capitolo 7), possiede quindi già entrambe le metà della corrispondenza che il pepper avrebbe dovuto proteggere, senza bisogno di alcun dump del database [5]. Il Capitolo 12 discute la mitigazione, una chiave di ricerca derivata con HMAC invece che con un digest semplice.

11.2 Costo di un attacco alle password

I progettisti di Argon2 esprimono deliberatamente la sicurezza dell'algoritmo come prodotto tra area e tempo, con fattori di riduzione adimensionali rispetto a un'implementazione a memoria piena, e RFC 9106, sezione 7, eredita lo stesso impianto: nessun tempo di attacco assoluto, nessun costo in dollari [37]. Il motivo è che un numero assoluto dipenderebbe da ipotesi sull'hardware dell'attaccante che non si possono fissare con certezza. Il conto che segue va quindi letto come un ordine di grandezza dichiaratamente approssimativo, non come una previsione: conviene dirlo prima di darlo, non dopo.

Nessun benchmark pubblico misura esattamente i parametri di RAM-USB, $m = 47104$, $t = 2$, $p = 1$. L'unico benchmark GPU pubblicato per Argon2, hashcat 7.0.0 (agosto 2025, modalità 34000), dichiara 1703 hash al secondo su una NVIDIA RTX 4090, ma con $m = 65536$ (64 MiB) e $t = 3$ [84]: più memoria e un passaggio in più dei nostri, il che renderebbe la velocità reale contro la configurazione di RAM-USB maggiore di quella misurata lì. L'unico studio accademico trovato, Eum et al. [85], misura invece a $m = 1024$ (1 MiB), quarantasei volte meno memoria: nessuno dei due è quindi un ancoraggio diretto, e il conto seguente parte perciò da un limite fisico derivato analiticamente, non da un benchmark.

Il tempo necessario a esaurire lo spazio è il rapporto tra il numero di candidati e la velocità con cui l'attaccante li verifica:

$$T = \frac{V(L)}{R},$$

dove R è il numero di tentativi al secondo che il suo hardware sostiene. Il numeratore dipende da come la password è stata scelta, il denominatore da come viene verificata, ed è su quest'ultimo che si concentra la strategia difensiva del sistema.

Essendo Argon2id memory-hard, ogni tentativo deve muovere attraverso la memoria almeno due volte i propri 46 MiB, quindi la banda dell'acceleratore fissa un tetto massimo al numero di tentativi al secondo, indipendentemente da quanto sia efficiente l'implementazione.

Applichiamo quel tetto alla più grande infrastruttura documentata. Il cluster Stargate di Abilene contava 201.600 acceleratori Blackwell operativi al 18 agosto 2026 [86], e NVIDIA dichiara 576 TB/s di banda per ogni rack da 72 GPU, cioè 8 TB/s ciascuna [87]. Dividendo la banda complessiva per il traffico di un tentativo si ottengono al più $1,7 \cdot 10^{10}$ tentativi al secondo.

A quel ritmo i $6,27 \cdot 10^{15}$ candidati di una password di otto caratteri si esaurirebbero in poco più di quattro giorni, in media due. Il tetto considera però la sola banda, e ignora sia il costo aritmetico della funzione di mescolamento sia la latenza degli accessi dipendenti dai dati, che nelle implementazioni reali mordono prima: il valore ottenuto è quindi un limite superiore alla velocità dell'attaccante, e i quattro giorni un limite inferiore al tempo che gli servirebbe.

Il conto presuppone inoltre che l'attaccante disponga del pepper oltre che del database, cioè che abbia compromesso il processo di Database-Vault e non soltanto i suoi dati: senza quel valore nessun tentativo è calcolabile. Il salt, che invece è nel record e quindi in suo possesso, non rallenta la singola ricerca ma la costringe a essere mirata, perché nessun tentativo vale per due utenti insieme: i quattro giorni sono il costo per una password, non per l'intero archivio.

Il confronto con una password più lunga mostra dove stia davvero la difesa. Con dodici caratteri sullo stesso alfabeto i candidati diventano $5,4 \cdot 10^{23}$, e lo stesso cluster impiegherebbe circa un milione di anni: quattro caratteri in più spostano il risultato di otto ordini di grandezza, mentre raddoppiare i passaggi di Argon2id si limiterebbe a raddoppiare il tempo.

11.3 Isolamento dei fallimenti

RNF-REL-01 richiede che la compromissione isolata di un componente non si propaghi agli altri. Questa sezione la verifica componente per componente, chiedendo per ciascuno cosa un attaccante otterrebbe davvero: i limiti già noti del sistema, distinti da questa domanda, restano nel Capitolo 12.

Entry-Hub compromesso vede solo ciò che Entry-Hub stesso può raggiungere, cioè Security-Switch, e solo con l'identità di Entry-Hub: anche disponendo del suo certificato, Network-Manager e Database-Vault rifiutano la connessione, perché le loro regole di raggiungibilità non includono più Entry-Hub tra i mittenti ammessi (Capitolo 7).

Il confinamento vale, più in generale, su due livelli indipendenti insieme, quello di rete e quello di identità: un nodo che detenga il tag di accesso temporaneo può ancora aprire una connessione TCP verso la porta amministrativa di Storage-Service invece che verso la sua SFTP, perché la regola di rete

non distingue le porte, ma la verifica del campo `organization` la rifiuta comunque. Anche dove il primo livello è più permissivo del necessario, il secondo tiene da solo.

Storage-Service compromesso resta confinato dal proprio chroot alla sola directory di sistema che gli appartiene (sezione 6.2): non vede né può elencare le directory degli altri utenti, e non ha comunque mai visto il contenuto in chiaro di nessun backup, perché quel contenuto non lascia mai il dispositivo dell'utente cifrato.

Database-Vault compromesso è lo scenario più costoso, non a caso lo stesso analizzato nella sezione 11.2: un attaccante che ne controlli il processo ottiene il pepper e può leggere le email cifrate insieme alla master key. Anche in quello scenario, però, non ottiene i file di backup degli utenti: Database-Vault non ha mai avuto accesso al filesystem di Storage-Service, solo alla facoltà di chiedergli, via mTLS, di creare un account (Capitolo 4).

11.4 Costo delle scelte architetturali

I capitoli precedenti pagano ciascuno un prezzo per una proprietà specifica, dichiarato dove viene speso. La Tabella 11.1 li raccoglie in un solo bilancio: l'argomento di ciascuna riga resta dove è già stato fatto, qui si somma solo il conto.

Tabella 11.1: Costo di ogni scelta architetturale rispetto alla proprietà che garantisce.

Scelta	Proprietà garantita	Prezzo pagato
Separazione in quattro processi	Rivalidazione indipendente, isolamento dei fallimenti	Latenza per hop, nessuno strato saltabile (Capitolo 4)
mTLS su ogni comunicazione interna	Autenticazione reciproca	Un certificato per componente, da emettere e rinnovare (sezione 4.4)
CA privata unica	Fiducia verificabile end-to-end	Un punto singolo di fiducia (sezione 4.4)
Rete mesh come confine di raggiungibilità	Storage-Service invisibile ai non autenticati	Dipendenza dal coordinatore, RISK-04 (Capitolo 7)
Argon2id con parametri alti	Resistenza al brute force offline	Memoria e latenza a ogni login (sezione 11.2)
Backup su filesystem via SFTP	Isolamento chroot come oggetto di studio	Complessità di ST-F-06..ST-F-11 (Capitolo 6)

Nessuna proprietà della colonna centrale è gratuita: ciascuna è comprata da un costo specifico e nominato nella colonna a destra, mai da un costo generico o non dichiarato. Non è un'ammissione di debolezza, ma l'inventario che un sistema attento alla sicurezza deve al proprio lettore: dire cosa costa, non solo cosa garantisce.

12

Conclusioni

12.1 Risultati raggiunti

Il Capitolo 1 apre sui tre principi che la SRS pone a fondamento del progetto, zero-knowledge, zero-trust e defense-in-depth [1], e sull'obiettivo dichiarato: non competere con soluzioni commerciali, ma costituire un caso di studio approfondito sulla progettazione sicura di sistemi di backup distribuiti, in cui la correttezza e la trasparenza della progettazione contano più della velocità di consegna o della copertura delle funzionalità.

I dieci requisiti utente elencati nel capitolo 3 sono, con una sola eccezione, coperti da requisiti di sistema implementati e tracciati fino al codice. Un utente può registrarsi fornendo solo email, password e chiave pubblica SSH (RU-01), autenticarsi nelle sessioni successive (RU-02), caricare e recuperare backup cifrati lato client da qualunque luogo (RU-03, RU-07), farlo solo dopo essersi autenticato (RU-04), con la garanzia che né la password né il contenuto dei backup siano mai leggibili da un amministratore (RU-05, RU-06), in uno spazio isolato da quello degli altri utenti (RU-08). Un amministratore può osservare la salute e le prestazioni del sistema senza mai toccare i dati dei singoli utenti (RU-10).

L'unica eccezione è RU-09, la garanzia che nessuno possa modificare o cancellare un backup: è stata dichiarata fuori perimetro fin dalla SRS, per i tempi stretti del progetto piuttosto che per una scelta di design, ed è ripresa nella prossima sezione.

Le cinque sequenze d'uso che attraversano il sistema, registrazione, login, backup, restore e consultazione delle metriche, non sono rimaste un esercizio sulla carta: sono state eseguite end-to-end sul deployment reale su Proxmox, non solo sullo stack Docker di sviluppo. Il 24 agosto 2026, per esempio, Security-Switch è stato arrestato e riavviato con il proprio token di bootstrap già consumato, ed è tornato in funzione riutilizzando l'identità mTLS persistita su disco invece di richiederne una nuova [5]: un dettaglio piccolo, ma è esattamente il tipo di comportamento che una verifica solo teorica non avrebbe potuto confermare.

12.2 Limiti noti

La SRS accetta esplicitamente quattro rischi come costo del rilascio nei tempi previsti, non come sviste [1]. Dichiararli qui, uno per uno, è la stessa disciplina di onestà già applicata nel Capitolo 11 al bilancio

delle scelte architetturali.

RISK-01 è RU-09 stesso: nessun requisito di sistema impedisce a un amministratore di modificare o cancellare un backup. Resta un miglioramento desiderabile, non ancora costruito.

RISK-02 riguarda la master key: risiede in una variabile d'ambiente, senza una procedura di backup (DV-F-18) né di rotazione (DV-F-19), entrambe requisiti "dovrebbe" e non "deve". La sua perdita comporterebbe la perdita irreversibile dell'accesso a tutti i dati cifrati; la sua compromissione romperebbe la garanzia zero-knowledge per ogni utente.

RISK-03 riguarda il client, pensato per girare nativamente sulla macchina dell'utente e non come container Docker. Containerizzarlo è stato considerato e scartato per ora: un container non vede percorsi arbitrari dell'host, come il Desktop dell'utente, a meno di un bind mount esplicito, e l'insieme dei file da salvare si sceglie liberamente al momento del backup, non è noto in anticipo come per ogni altro componente. L'opzione che perde meno isolamento, se il tema fosse ripreso, è un bind mount per singola invocazione, limitato alla sola cartella oggetto del backup.

RISK-04, la dipendenza circolare della risoluzione DNS di Network-Manager dal proprio stesso MagicDNS, è già discussa nella sezione 10.5: qui vale solo ricordare che è un rischio accettato per scelta esplicita, non un difetto scoperto in seguito.

12.3 Sviluppi futuri

12.3.1 Mitigazione dell'enumerazione degli indirizzi email

La sezione 11.1 analizza RISK-05: la registrazione rivela, tramite il codice HTTP restituito, se un indirizzo è già registrato. Il comportamento ideale risponderebbe in modo identico in entrambi i casi, per esempio sempre con un 202, mentre l'esito reale viaggierebbe fuori banda, in un'email di conferma inviata all'indirizzo indicato: solo chi possiede davvero quella casella riceverebbe il seguito, mentre chi sonda l'endpoint riceverebbe sempre la stessa risposta. Un effetto collaterale positivo scomparirebbe insieme al problema: senza un account attivato dalla conferma, sparirebbe anche l'occupazione abusiva di un indirizzo altrui.

Il costo non va taciuto: servirebbe una capacità di invio email che il sistema oggi non ha, un client SMTP, token di conferma con scadenza, e una deliverability affidabile. Cambierebbe inoltre UC-01, perché la pre-auth key oggi viaggia dentro la risposta 201 della registrazione, mentre con una registrazione in due tempi dovrebbe essere consegnata solo dopo la conferma.

12.3.2 Le due regole della policy delle password

Il sottosezione 5.2.1 misura che, sotto scelta uniforme, il vincolo di almeno tre categorie di carattere vale meno di un decimo di bit di entropia, mentre un carattere aggiuntivo ne vale 6,57: sotto quell'ipotesi la regola di composizione è ridondante per costruzione, perché una password davvero casuale usa già più categorie da sé. Il suo scopo reale, però, non è aumentare l'entropia, ma vincolare la scelta di una persona, e lì il suo valore non è misurabile senza la distribuzione reale delle password scelte dagli utenti.

Restano due direzioni da confrontare: tenere entrambe le regole così come sono, oppure eliminare il vincolo di composizione e alzare la lunghezza minima, la strada indicata dalle linee guida NIST correnti

[88], che raccomandano ai verificatori di non imporre regole di composizione e portano il minimo per una password usata da sola a 15 caratteri. Anche la seconda strada ha però un costo da non nascondere: senza una lista di password compromesse note, togliere il vincolo di composizione rimuove una protezione debole senza sostituirla con una forte, e una password come `password` tornerebbe accettabile.

12.3.3 Hash con chiave per la ricerca dell'email

Il sezione 11.1 analizza anche RISK-06: la chiave di ricerca dell'email è oggi uno SHA-256 senza chiave, veloce da calcolare su uno spazio di indirizzi piccolo e indovinabile. Il comportamento ideale deriverebbe quella chiave con una funzione dotata di chiave, HMAC-SHA256 sotto il pepper, e non con la concatenazione ingenua che nel percorso della password basta solo perché lì interviene Argon2id. Un pepper è compatibile con la ricerca, a differenza di un salt: essendo un unico segreto globale, il digest resta deterministico e la ricerca al login continua a funzionare identica. Il guadagno sarebbe che una copia del database, da sola, non permetterebbe più di verificare se un indirizzo sia registrato, la stessa difesa che il sezione 11.2 rivendica già per la password. Non risolverebbe invece RISK-05, che resta un attacco online contro l'API: sono due superfici diverse.

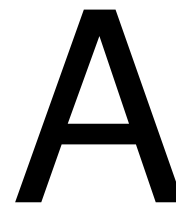
Il costo dichiarato dalla SRS per questa modifica presuppone però una reversibilità che oggi non esiste ancora nel codice, e va corretto qui. La SRS descrive ogni `email_hash` come ricostruibile decifrando l'email con la master key, dato che è cifrata e non solo hashata; il record salvato, insieme alla master key, contiene sì in linea di principio tutto il necessario.

`encryption.go`, però, dichiara oggi solo `EncryptEmail`, non la funzione inversa: nessun percorso del codice decifra mai quella colonna. La reversibilità è quindi, per ora, una proprietà dei dati salvati, non ancora del sistema: la migrazione dovrebbe prima scrivere quel percorso di decifratura mancante, non solo invalidare e ricalcolare gli hash esistenti.

Il costo resta comunque reale anche a percorso scritto: il pepper diventerebbe portante anche per la ricerca dei record, non solo per la verifica delle password, spostando la conseguenza della sua perdita da "non si verificano più le password" a "non si trovano più i record", da valutare accanto a RISK-02.

12.3.4 Altri requisiti "dovrebbe" non implementati

Oltre a DV-F-18 e DV-F-19, già discussi come RISK-02, restano "dovrebbe" e non ancora costruiti il limite di quota per utente (ST-F-14) e la replica automatica dei backup su CephFS (ST-F-15), entrambi rimandati nella SRS come miglioramenti desiderabili piuttosto che vincoli del progetto. Resta inoltre da fare, sul piano della verifica piuttosto che dell'implementazione, un test di carico e di stress che misuri il sistema oltre le condizioni già esercitate in questa tesi.



Elenco completo dei requisiti

Le tabelle seguenti raccolgono i requisiti di sistema funzionali, non funzionali e di dominio, organizzati secondo la stessa struttura dell'SRS originale. [1]

Il capitolo 3 discute per intero i requisiti utente, i quali sono gli unici a non essere inclusi nelle tabelle sottostanti.

A.1 Requisiti di sistema funzionali

A.1.1 Client

Tabella A.1: Requisiti funzionali del Client.

ID	Requisito
CL-F-01	Deve generare autonomamente una coppia di chiavi SSH (pubblica/privata) prima della registrazione, senza mai trasmettere la chiave privata ad alcun componente del sistema.
CL-F-02	Su comando dell'utente, deve inviare a Entry-Hub, durante la registrazione (<code>POST /api/users</code>), solo la chiave pubblica SSH, insieme a email e password.
CL-F-03	Su comando dell'utente, deve ripetere il login (<code>POST /api/login</code>) prima della scadenza del grant ACL di 12 ore, per mantenere la continuità di accesso a Storage-Service.
CL-F-04	Deve configurare e avviare Tailscale usando la pre-auth key ricevuta nella risposta di registrazione, per unirsi alla rete mesh privata.
CL-F-05	Deve risolvere Storage-Service tramite MagicDNS sulla rete mesh, senza fare affidamento su indirizzi IP statici.
CL-F-06	Su comando dell'utente, deve invocare <code>restic backup</code> verso Storage-Service via SFTP sulla porta TCP 2222 (ST-F-03), autenticandosi con la chiave privata SSH generata in CL-F-01.

ID	Requisito
CL-F-07	Su comando dell'utente, deve invocare restic restore verso Storage-Service via SFTP sulla porta TCP 2222 (ST-F-03), usando lo stesso metodo di autenticazione di CL-F-06.
CL-F-08	Deve gestire i codici di errore HTTP restituiti da Entry-Hub (400/401/403/500/502/503/504) senza esporre dettagli interni del sistema all'utente finale.
CL-F-09	Su comando dell'utente, deve validare localmente email, password e — solo in fase di registrazione — la chiave pubblica SSH, usando le stesse regole imposte da Entry-Hub (EH-F-04 per la registrazione, EH-F-05 per il login), prima di inviare POST /api/users o POST /api/login ; in caso di fallimento della validazione locale, non deve trasmettere la richiesta.
CL-F-10	Deve ricavare l'endpoint di login di Entry-Hub da un valore di configurazione distinto da quello di registrazione, e deve usare per il login un nome coperto dal certificato che Entry-Hub presenta.
CL-F-11	Deve fissare la chiave host SSH di Storage-Service: registra la chiave ricevuta nella risposta alla registrazione (ST-F-16) prima del primo backup, la presenta a ssh come host noto e interrompe la connessione se la chiave presentata differisce.

A.1.2 Entry-Hub

Tabella A.2: Requisiti funzionali di Entry-Hub.

ID	Requisito
EH-F-01	Deve esporre un endpoint HTTPS pubblico di health-check (GET /api/health), con certificati firmati dalla CA pubblica Let's Encrypt.
EH-F-02	Deve esporre un endpoint HTTPS pubblico per la registrazione degli utenti (POST /api/users), con certificati firmati dalla CA pubblica Let's Encrypt.
EH-F-03	Deve esporre un endpoint HTTPS (POST /api/login) per l'autenticazione degli utenti registrati, servito su un listener separato raggiungibile solo dalla rete mesh privata (non da internet, a differenza del listener di EH-F-01/EH-F-02), con lo stesso certificato firmato dalla CA pubblica Let's Encrypt presentato da EH-F-01/EH-F-02 — non mTLS; cambia solo la raggiungibilità di rete.
EH-F-04	/api/users deve accettare JSON e validare: presenza dei campi email, password e chiave pubblica SSH; dimensione del payload entro un limite definito; assenza di campi aggiuntivi inattesi; formato dell'email (RFC 5322); lunghezza della password tra 8 e 128 caratteri; complessità della password (almeno 3 categorie di caratteri tra minuscole, maiuscole, cifre, simboli); correttezza formale della chiave pubblica SSH.

ID	Requisito
EH-F-05	/api/login deve accettare JSON e validare: presenza dei campi email e password; dimensione del payload entro un limite definito; assenza di campi aggiuntivi inattesi; formato dell'email (RFC 5322); lunghezza della password tra 8 e 128 caratteri; complessità della password (almeno 3 categorie di caratteri).
EH-F-06	In caso di fallimento della validazione deve: rispondere con HTTP 400 (Bad Request) senza specificare quale problema sia stato riscontrato; registrare nel log il problema trovato senza identificare l'utente; non inoltrare la richiesta ad alcun altro servizio interno.
EH-F-07	In caso di validazione riuscita deve: registrare nel log l'esito della validazione senza identificare l'utente; inoltrare la richiesta a Security-Switch via mTLS; verificare che il certificato provenga da un Security-Switch; verificare la validità del certificato X.509.
EH-F-08	Deve inoltrare all'utente la risposta ricevuta da Security-Switch.
EH-F-09	Deve mappare gli errori interni sui codici HTTP 400/401/500/502/503, restituendo al client messaggi sanificati e mantenendo i log dettagliati solo internamente.
EH-F-10	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (metrics/Entry-Hub), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
EH-F-11	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.

A.1.3 Security-Switch

Tabella A.3: Requisiti funzionali di Security-Switch.

ID	Requisito
SS-F-01	Deve accettare solo connessioni mTLS da client che soddisfino: campo organization uguale a EntryHub ; certificato X.509 valido; accesso alla rete mesh privata.
SS-F-02	Deve ri-validare l'input ricevuto, indipendentemente dalla validazione già effettuata da Entry-Hub.
SS-F-03	In caso di fallimento della validazione deve: rispondere con HTTP 400 (Bad Request) senza specificare quale problema sia stato riscontrato; registrare nel log il problema trovato senza identificare l'utente; non inoltrare la richiesta ad alcun altro servizio interno.
SS-F-04	In caso di validazione riuscita deve: registrare nel log l'esito della validazione senza identificare l'utente; inoltrare la richiesta a Database-Vault via mTLS, verificando che: il certificato provenga da un Database-Vault; il certificato X.509 sia valido.

ID	Requisito
SS-F-05	Dopo la conferma di autenticazione riuscita da Database-Vault, deve richiedere a Network-Manager (via mTLS) di concedere a quell'utente l'accesso a Storage-Service per 12 ore.
SS-F-06	Deve mappare gli errori sui codici HTTP 400/401/403/500/502/504.
SS-F-07	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (metrics/Security-Switch), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
SS-F-08	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.
SS-F-09	Dopo la conferma di registrazione riuscita da Database-Vault, deve richiedere a Network-Manager (via mTLS) di creare un utente Headscale dedicato e generare una pre-auth key per il nuovo account, includendo poi quella chiave nella risposta a Entry-Hub.

A.1.4 Database-Vault

Tabella A.4: Requisiti funzionali di Database-Vault.

ID	Requisito
DV-F-01	Deve accettare solo connessioni mTLS da client che soddisfino: campo organization uguale a SecuritySwitch ; certificato valido; accesso alla rete mesh privata.
DV-F-02	Deve ri-validare l'input ricevuto, indipendentemente dalla validazione già effettuata da Security-Switch.
DV-F-03	Deve calcolare l'hash SHA-256 dell'email per l'indicizzazione e come chiave primaria, senza mai registrare nel log l'email in chiaro.
DV-F-04	Deve cifrare l'email dell'utente: derivare una chiave di cifratura per record dalla master key con HKDF-SHA256 e un salt casuale a 16 byte, poi cifrare l'email con AES-256-GCM usando quella chiave derivata e un nonce casuale a 12 byte.
DV-F-05	La master key dovrebbe provenire da una sorgente configurabile, con validazione della lunghezza (32 byte).
DV-F-06	Deve mantenere un pepper come variabile d'ambiente.
DV-F-07	Deve calcolare l'hash della password con Argon2id: memoria 47104 KiB (46 MiB), 2 iterazioni, parallelismo 1, output a 32 byte, usando un salt casuale per record e il pepper (DV-F-06).
DV-F-08	Deve salvare il record utente in una transazione atomica.
DV-F-09	Deve richiedere a Storage-Service la creazione dell'utente POSIX univoco sul server, con username user<xxxxxx> (xxxxxx : 6 caratteri casuali da un alfabeto base-36), e attenderne la risposta.

ID	Requisito
DV-F-10	Se la creazione dell'utente POSIX fallisce, deve eliminare l'utente dal database e informare Security-Switch che la registrazione è fallita.
DV-F-11	Dopo aver creato il record utente e l'utente POSIX, deve informare Security-Switch che l'utente è stato registrato.
DV-F-12	Deve rifiutare (HTTP 409) le registrazioni con un'email o una chiave SSH già esistenti, senza fornire dettagli sull'errore.
DV-F-13	Durante il login, deve recuperare il salt associato all'email tramite l'hash SHA-256 dell'email (DV-F-03).
DV-F-14	Deve ricalcolare Argon2id sulla password ricevuta usando il salt recuperato e il pepper, confrontando il risultato con l'hash memorizzato.
DV-F-15	Deve rispondere con lo stesso codice HTTP 401 sia per un'email inesistente sia per una password errata, senza distinguere i due casi né nella risposta né nel log.
DV-F-16	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (metrics/Database-Vault), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
DV-F-17	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.
DV-F-18	Dovrebbe esistere una procedura di backup della master key.
DV-F-19	Dovrebbe esistere una procedura di rotazione della master key.
DV-F-20	In caso di fallimento della validazione deve: rispondere con HTTP 400 (Bad Request) senza specificare quale problema sia stato riscontrato; registrare nel log il problema trovato senza identificare l'utente; non inoltrare la richiesta ad alcun altro servizio interno.

A.1.5 Storage-Service

Tabella A.5: Requisiti funzionali di Storage-Service.

ID	Requisito
ST-F-01	Deve accettare connessioni mTLS solo da client che soddisfino: campo organization uguale a DatabaseVault ; certificato valido; accesso alla rete mesh privata.
ST-F-02	Deve fornire upload/download di file già cifrati lato client, senza mai elaborare contenuto in chiaro.
ST-F-03	L'accesso deve avvenire esclusivamente via SFTP sulla porta TCP 2222, autenticato con la chiave pubblica SSH registrata dall'utente.
ST-F-04	Deve vietare esplicitamente qualsiasi forma di connessione SSH diversa da SFTP.
ST-F-05	Ogni utente deve disporre di uno spazio di storage isolato, non accessibile dagli altri utenti.

ID	Requisito
ST-F-06	Su richiesta di Database-Vault via mTLS, deve creare sul sistema un utente POSIX con username <code>user<xxxxxx></code> (<code>xxxxxx</code> : 6 caratteri casuali da un alfabeto base-36).
ST-F-07	Deve garantire che l'utente POSIX non possa mai uscire dalla propria directory.
ST-F-08	L'account POSIX creato non deve avere una home directory tradizionale né una shell interattiva; l'unico spazio scrivibile è la sottodirectory dedicata all'interno del <code>chroot</code> .
ST-F-09	La configurazione <code>sshd</code> di Storage-Service deve avere <code>PasswordAuthentication no</code> e <code>PermitRootLogin no</code> , indipendentemente dal fatto che gli account creati non abbiano una password impostata.
ST-F-10	A seguito della creazione, riuscita o fallita, dell'utente POSIX, deve riportare l'esito a Database-Vault.
ST-F-11	Ad ogni tentativo di connessione SFTP dell'utente, deve recuperare la chiave pubblica corrente dell'utente da Database-Vault tramite <code>AuthorizedKeysCommand</code> .
ST-F-12	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (<code>metrics/Storage-Service</code>), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
ST-F-13	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.
ST-F-14	Dovrebbe imporre limiti di quota per utente.
ST-F-15	Dovrebbe garantire: replica automatica dei dati; tolleranza ai guasti per almeno un nodo; consistenza dei dati; possibilità di espansione tramite l'aggiunta di nuovi nodi senza interrompere il servizio.
ST-F-16	Deve restituire la propria chiave pubblica host SSH nella risposta alla richiesta di creazione dell'utente POSIX proveniente da Database-Vault (ST-F-10), affinché possa essere inoltrata al client e fissata da questo (CL-F-11).

A.1.6 Network-Manager

Tabella A.6: Requisiti funzionali di Network-Manager.

ID	Requisito
NM-F-01	Deve garantire che solo Entry-Hub possa contattare Security-Switch.
NM-F-02	Deve garantire che solo Security-Switch e Storage-Service possano contattare Database-Vault.
NM-F-03	Deve garantire che solo Security-Switch possa contattare Network-Manager.
NM-F-04	Deve garantire che tutti i componenti interni della rete, eccetto gli utenti, possano contattare la Certificate-Authority sulla rete mesh.
NM-F-05	Deve garantire che solo gli utenti autenticati possano vedere e contattare Storage-Service.

ID	Requisito
NM-F-06	Deve garantire che gli utenti registrati ma non autenticati possano vedere e contattare solo Entry-Hub.
NM-F-07	Deve garantire che gli utenti registrati e autenticati possano vedere e contattare solo Entry-Hub e Storage-Service.
NM-F-08	Su richiesta di Security-Switch, a seguito di una registrazione riuscita, deve creare un utente Headscale dedicato e generare una pre-auth key a breve durata per il nuovo account.
NM-F-09	Dopo un login riuscito, su richiesta di Security-Switch, deve assegnare al nodo dell'utente il tag ACL che abilita la raggiungibilità verso Storage-Service, registrando una scadenza a 12 ore da quel momento.
NM-F-10	Deve verificare periodicamente le scadenze registrate e rimuovere il tag ACL dai nodi scaduti, automaticamente e senza intervento manuale.
NM-F-11	La scadenza di ogni grant deve essere persistita, non mantenuta solo in memoria, per non perdere lo stato in caso di riavvio di Network-Manager.
NM-F-12	La creazione di pre-auth key e la gestione dei tag ACL devono essere ristrette specificamente a Network-Manager, verificato via mutual TLS (campo organization uguale a NetworkManager).
NM-F-13	La pre-auth key serve unicamente a registrare il nodo come membro della mesh; da sola, non concede raggiungibilità verso Storage-Service.
NM-F-14	L'endpoint di coordinamento di Headscale è deliberatamente raggiungibile dalla rete pubblica (idealmente ospitato su un proprio VPS con indirizzo pubblico).
NM-F-15	Deve configurare MagicDNS con un dominio di base dedicato, in modo che Storage-Service sia risolvibile da tutti i nodi della mesh tramite un nome stabile anziché un IP.
NM-F-16	Il nodo mesh di Network-Manager deve accettare la configurazione DNS distribuita da Headscale (MagicDNS), in modo che le proprie chiamate in uscita verso Certificate-Authority e MQTT-Broker risolvano quegli hostname sulla mesh, anziché ricadere su un percorso di rete privo di equivalente in produzione.
NM-F-17	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (metrics/Network-Manager), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
NM-F-18	Le metriche non devono mai contenere dati personali degli utenti, solo statistiche aggregate.
NM-F-19	Deve configurare MagicDNS con un record che risolva il nome pubblico di Entry-Hub sul suo indirizzo mesh, così che un membro della rete che contatta il listener di login (EH-F-03) usi il nome coperto dal certificato di Entry-Hub.
NM-F-20	Deve garantire che solo i servizi che pubblicano metriche (Entry-Hub, Security-Switch, Database-Vault, Storage-Service, Network-Manager, Certificate-Authority) e Metrics-Collector possano contattare MQTT-Broker.
NM-F-21	Deve garantire che solo Grafana possa contattare Metrics-Collector.

A.1.7 Certificate-Authority

Tabella A.7: Requisiti funzionali di Certificate-Authority.

ID	Requisito
CA-F-01	Deve garantire che i componenti che presentano certificati per mTLS siano realmente chi dichiarano di essere.
CA-F-02	Deve garantire l'emissione, la rotazione, la revoca e la verifica dei certificati mTLS.
CA-F-03	Deve pubblicare le metriche ogni minuto, e solo allora, sul proprio topic MQTT dedicato (<code>metrics/Certificate-Authority</code>), via mTLS, verificando che: il certificato provenga da un MQTT-Broker; il certificato X.509 sia valido.
CA-F-04	Deve accettare un bootstrap token monouso, distribuito fuori banda a ciascun servizio, per l'emissione del certificato iniziale; i rinnovi successivi devono avvenire tramite il certificato mTLS corrente, non tramite il token.

A.1.8 Sistema di monitoraggio

Tabella A.8: Requisiti funzionali del sistema di monitoraggio (MQTT-Broker / Metrics-Collector / TimescaleDB / Grafana).

ID	Requisito
MT-F-01	Metrics-Collector può leggere solo <code>metrics/*</code> .
MT-F-02	Metrics-Collector deve scartare le metriche il cui campo <code>service</code> non corrisponde al topic da cui provengono.
MT-F-03	Le metriche devono essere memorizzate come hypertable TimescaleDB, con retention automatica di 30 giorni e compressione dopo 7 giorni.
MT-F-04	Devono esistere dashboard Grafana per tempo di risposta, throughput e connessioni attive.

A.1.9 Infrastruttura di rete e PKI

Tabella A.9: Requisiti funzionali di rete e PKI.

ID	Requisito
PKI-F-01	Ogni servizio deve autenticarsi mutuamente con certificati X.509 emessi da una CA valida.
PKI-F-02	Ogni servizio deve verificare il campo <code>organization</code> del certificato, non solo la sua validità.
PKI-F-03	Dovrebbe esistere una procedura di rotazione e revoca dei certificati.

ID	Requisito
NET-F-01	La comunicazione tra servizi deve avvenire sulla rete privata; gli endpoint pubblici di Entry-Hub e l'endpoint di coordinamento Headscale di Network-Manager (NM-F-14) sono le uniche superfici deliberatamente esposte pubblicamente dal sistema.
NET-F-02	Il TLS utilizzato deve essere in versione 1.3.

A.2 Requisiti non funzionali

A.2.1 Di prodotto

Tabella A.10: Requisiti non funzionali di prodotto.

ID	Requisito
RNF-SEC-01	Zero-knowledge: nessun componente server-side accede mai al contenuto dei file di backup in chiaro, poiché la cifratura avviene lato client prima della trasmissione. Questo non si estende alle credenziali di accesso: email e password transitano, cifrate (TLS/mTLS), attraverso Entry-Hub, Security-Switch e Database-Vault per la validazione e l'hashing, pur senza mai essere persistite o registrate nei log in chiaro (cfr. DV-F-03, RD-01).
RNF-SEC-02	Zero-trust: nessun servizio deve fidarsi implicitamente dei dati ricevuti da un altro, anche se autenticato via mTLS.
RNF-SEC-03	Defense-in-depth: ogni livello ri-valida l'input in modo indipendente.
RNF-SEC-04	Tutta la comunicazione tra servizi deve usare mTLS, senza eccezioni.
RNF-REL-01	Il sistema deve tollerare la compromissione isolata di un singolo componente, senza che questa si propaghi agli altri.
RNF-PERF-01	La latenza delle richieste HTTP deve essere tracciata (p50/p95/p99).
RNF-USA-01	I messaggi di errore devono essere comprensibili e correttamente categorizzati per codice HTTP.
RNF-MAINT-01	Ogni servizio deve poter essere isolato, ri-certificato e riavviato individualmente senza impattare gli altri.

A.2.2 Organizzativi

Tabella A.11: Requisiti non funzionali organizzativi.

ID	Requisito
RNF-ORG-01	Linguaggio di implementazione: Go.
RNF-ORG-03	Licenza open-source MIT.
RNF-ORG-04	Piattaforma di deployment: Proxmox VE (KVM per Storage-Service, Database-Vault, Network-Manager; LXC per gli altri servizi).
RNF-ORG-05	Sviluppo ed esercizio garantiti su macOS e Linux (con Docker).

A.2.3 Esterni

Tabella A.12: Requisiti non funzionali esterni.

ID	Requisito
RNF-EXT-01	Poiché il sistema elabora dati personali (l'email), dovrebbe rispettare le normative sulla privacy applicabili (es. GDPR). Attualmente fuori perimetro.

A.3 Requisiti di dominio

Tabella A.13: Requisiti di dominio (RD).

ID	Requisito
RD-01	Qualsiasi nuovo componente introdotto in futuro non deve creare un percorso lungo il quale dati sensibili in chiaro attraversino o vengano registrati da un componente diverso dal client o dal componente strettamente necessario alla loro cifratura/decifratura.
RD-02	Le chiavi derivate (tramite HKDF) non devono mai essere persistite: qualsiasi nuovo requisito di memorizzazione di chiavi deve essere valutato rispetto a questo vincolo prima di essere accettato.
RD-03	Argon2id e AES-256-GCM sono vincoli tecnologici non negoziabili.
RD-04	Il principio di <i>fail-secure</i> si applica a ogni componente: in caso di incertezza sulla validità di una richiesta, il comportamento predefinito è negare l'accesso.

Bibliografia

- [1] Francesco Verrengia. RAM-USB software requirements specification. https://github.com/Verryx-02/RAM-USB/blob/main/docs/Software_Requirements_Specification.md, 2026.
- [2] Francesco Verrengia. RAM-USB contributing guidelines. <https://github.com/Verryx-02/RAM-USB/blob/main/CONTRIBUTING.md>, 2026.
- [3] Francesco Verrengia. RAM-USB: Software test plan. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/docs/Test_Plan.md, 2026.
- [4] Francesco Verrengia. RAM-USB: project README, "use of AI assistance". <https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/README.md>, 2026.
- [5] Francesco Verrengia. RAM-USB known issues (scratch cache). https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/docs/Known_Issues.md, 2026.
- [6] Francesco Verrengia. RAM-USB: make diagrams, rendering PlantUML sources on every change. <https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/Makefile#L1-L2>, 2026.
- [7] Scott Rose, Oliver Borchert, Stu Mitchell, e Sean Connelly. NIST special publication 800-207: Zero Trust Architecture. <https://doi.org/10.6028/NIST.SP.800-207>, 2020.
- [8] Shafi Goldwasser, Silvio Micali, e Charles Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989. <https://doi.org/10.1137/0218012>.
- [9] restic. restic: Fast, secure, efficient backup program. <https://github.com/restic/restic>, 2026.
- [10] Tarsnap. Tarsnap: online backups for the truly paranoid, cryptography. <https://www.tarsnap.com/crypto.html>, 2026.
- [11] Borg Collective. BorgBackup documentation. <https://borgbackup.readthedocs.io/en/stable/>, 2026.
- [12] Bitwarden. What encryption is used? <https://bitwarden.com/help/what-encryption-is-used/>, 2026.
- [13] Tresorit. Tresorit security. <https://tresorit.com/security>, 2026.
- [14] Juan Font. headscale: An open source, self-hosted implementation of the tailscale control server. <https://github.com/juanfont/headscale>, 2026.

- [15] Tailscale. How tailscale works. <https://tailscale.com/blog/how-tailscale-works>, 2026.
- [16] Barry W. Boehm. A spiral model of software development and enhancement. *IEEE Computer*, 21(5):61–72, 1988. <https://doi.org/10.1109/2.59>.
- [17] Francesco Verrengia. RAM-USB software requirements specification, v1.0: original wording of NM-F-12. https://github.com/Verryx-02/RAM-USB/blob/3ef723e07a0292337fc48a1cd2e8c952f2f2e2ee/docs/Software_Requirements_Specification.md?plain=1#L260, 2026.
- [18] Francesco Verrengia. RAM-USB: correct NM-F-12 wording and document how headscale’s CLI satisfies it. <https://github.com/Verryx-02/RAM-USB/commit/975be1f467873628532a807e852f086dba29a9ef>, 2026.
- [19] Francesco Verrengia. RAM-USB: rework NM-F-12/NM-F-14/NET-F-01 for a standalone public headscale and retire EH-F-12. <https://github.com/Verryx-02/RAM-USB/commit/f2d17f41668ae2c978c849ac308ac4fa52f7b409>, 2026.
- [20] Francesco Verrengia. RAM-USB contributing guidelines: Commit structure (section 2.2). <https://github.com/Verryx-02/RAM-USB/blob/7984a140e0ce7f6b3ea89470801c0d26ecbd4aa1/CONTRIBUTING.md?plain=1#L33-L101>, 2026.
- [21] Google. distroless: Language focused docker images, minus the operating system. <https://github.com/GoogleContainerTools/distroless>, 2026.
- [22] Murugiah Souppaya, John Morello, e Karen Scarfone. Application container security guide. <https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-190.pdf>, 2017. NIST Special Publication 800-190.
- [23] Michael Wager. Evaluating the security of containers through reduction of potentially vulnerable components. https://www.mwager.de/assets/component_reduction_paper.pdf, 2024. Hochschule Augsburg.
- [24] Josh Aas, Richard Barnes, Benton Case, Zakir Durumeric, Peter Eckersley, Alan Flores-López, J. Alex Halderman, Jacob Hoffman-Andrews, James Kasten, Eric Rescorla, Seth Schoen, e Brad Warren. Let’s Encrypt: An automated certificate authority to encrypt the entire web. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS ’19)*, pp. 2473–2487. ACM, 2019. <https://doi.org/10.1145/3319535.3363192>.
- [25] Francesco Verrengia. RAM-USB: decodeJSON and ValidateRegister. <https://github.com/Verryx-02/RAM-USB/blob/7c1e991f953a4c686e09f8142d0b3c1d9d64fc5f/pkg/validation/validation.go#L125-L173>, 2026.
- [26] Francesco Verrengia. RAM-USB: shared internal libraries. <https://github.com/Verryx-02/RAM-USB/tree/main/pkg>, 2026.

- [27] Tatu Ylönen e Sami Lehtinen. SSH file transfer protocol. <https://datatracker.ietf.org/doc/html/draft-ietf-secsh-filexfer-02>, 2001. Internet-Draft draft-ietf-secsh-filexfer-02, la revisione implementata da OpenSSH, mai divenuta RFC.
- [28] OWASP. Fail securely. https://owasp.org/www-community/Fail_securely, 2026.
- [29] Smallstep. certificates: A private certificate authority (x.509 & ssh) & acme server for secure automated certificate management. <https://github.com/smallstep/certificates>, 2026.
- [30] Timescale. TimescaleDB: A time-series database for high-performance real-time analytics. <https://github.com/timescale/timescaledb>, 2026.
- [31] Grafana Labs. Grafana: The open and composable observability and data visualization platform. <https://github.com/grafana/grafana>, 2026.
- [32] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. Tesi di Dottorato di Ricerca, University of California, Irvine, 2000. <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>.
- [33] Peter W. Resnick. Internet message format. <https://doi.org/10.17487/RFC5322>, 2008. RFC 5322.
- [34] Hugo Krawczyk e Pasi Eronen. HMAC-based extract-and-expand key derivation function (HKDF). <https://doi.org/10.17487/RFC5869>, 2010. RFC 5869.
- [35] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC. <https://doi.org/10.6028/NIST.SP.800-38D>, 2007. NIST Special Publication 800-38D.
- [36] Francesco Verrengia. RAM-USB: derivazione della chiave per record con HKDF-SHA256. <https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/encryption/encryption.go#L88-L109>, 2026.
- [37] Alex Biryukov, Daniel Dinu, Dmitry Khovratovich, e Simon Josefsson. Argon2 memory-hard function for password hashing and proof-of-work applications. <https://doi.org/10.17487/RFC9106>, 2021. RFC 9106.
- [38] OWASP. Password storage cheat sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html, 2026.
- [39] Francesco Verrengia. RAM-USB: salvataggio del record utente in transazione atomica. <https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/storage/storage.go#L111-L146>, 2026.
- [40] Francesco Verrengia. RAM-USB: rollback compensativo dopo un fallimento di storage-service. <https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/registration/registration.go#L157-L165>, 2026.

- [41] Francesco Verrengia. RAM-USB: esito unico del login per email inesistente e password errata. <https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/login/login.go#L160-L185>, 2026.
- [42] Francesco Verrengia. RAM-USB: verifica fittizia che pareggia i tempi di risposta del login. <https://github.com/Verryx-02/RAM-USB/blob/3a5b843c439fc62df99212b1d960d119dd6fe1ea/services/database-vault/internal/password/hash.go#L178-L193>, 2026.
- [43] Francesco Verrengia. RAM-USB: generation and local storage of the Restic repository password. <https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/user-client/internal/reposecret/reposecret.go#L51-L77>, 2026.
- [44] restic. restic documentation: references and repository design. https://restic.readthedocs.io/en/stable/100_references.html, 2026.
- [45] OpenBSD Project. `sshd_config(5)`: Openssh daemon configuration file. https://man.openbsd.org/sshd_config.5, 2026.
- [46] Francesco Verrengia. RAM-USB: Storage-service hardened `sshd_config`. https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/deployments/docker/storage-service/sshd_config, 2026.
- [47] Francesco Verrengia. RAM-USB: fail-secure exit paths of the `AuthorizedKeysCommand` binary. <https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/storage-service/cmd/authorized-keys-command/main.go#L97-L131>, 2026.
- [48] Francesco Verrengia. RAM-USB: Restic repository URL and SFTP transport command. <https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/user-client/internal/restic/restic.go#L74-L90>, 2026.
- [49] Francesco Verrengia. RAM-USB: mesh reachability rules published to Headscale. <https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/network-manager/internal/headscale/policy.go#L198-L336>, 2026.
- [50] Francesco Verrengia. RAM-USB: mesh identity and pre-auth key creation at registration. <https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/network-manager/internal/headscale/client.go#L198-L217>, 2026.
- [51] Francesco Verrengia. RAM-USB: assignment of the Storage-Service access tag after login. <https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/network-manager/internal/headscale/client.go#L251-L275>, 2026.
- [52] Francesco Verrengia. RAM-USB: SQLite schema of the permanent email-to-pre-auth-key-ID mapping. <https://github.com/Verryx-02/RAM-USB/blob/>

- cd4b9fc3b1f503d96040287d4849c8cc17f2ac/services/network-manager/internal/grants/meshusers.go#L20-L25, 2026.
- [53] Francesco Verrengia. RAM-USB: SQLite schema of the persisted access-grants table. <https://github.com/Verryx-02/RAM-USB/blob/cd4b9fc3b1f503d96040287d4849c8cc17f2ac/services/network-manager/internal/grants/store.go#L87-L94>, 2026.
- [54] Francesco Verrengia. RAM-USB: periodic revocation of expired access grants. <https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/services/network-manager/internal/grants/sweep.go#L54-L88>, 2026.
- [55] Headscale. Headscale FAQ: can I use headscale and tailscale on the same machine? <https://headscale.net/stable/about/faq/>, 2026.
- [56] Francesco Verrengia. RAM-USB: mTLS enforcement on Headscale's administrative path. <https://github.com/Verryx-02/RAM-USB/blob/9576f3979d397f4acd57c7012c792c01e3f73095/deployments/docker/headscale/nginx.conf#L110-L129>, 2026.
- [57] Jason Weil, Victor Kuarsingh, Chris Donley, Christopher Liljenstolpe, e Marla Azinger. IANA-reserved IPv4 prefix for shared address space. <https://doi.org/10.17487/RFC6598>, 2012. RFC 6598, BCP 153.
- [58] Eclipse Foundation. Eclipse Mosquitto: an open source MQTT broker. <https://github.com/eclipse-mosquitto/mosquitto>, 2026.
- [59] Francesco Verrengia. RAM-USB: shape of the aggregated metrics payload. <https://github.com/Verryx-02/RAM-USB/blob/dd0302834b0222ec9ef1899a01735d16725c5137/pkg/metrics/payload.go#L50-L57>, 2026.
- [60] Andrew Banks e Rahul Gupta. MQTT version 3.1.1 plus errata 01. <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/errata01/os/mqtt-v3.1.1-errata01-os-complete.html>, 2015.
- [61] Francesco Verrengia. RAM-USB: hypertable, retention, and compression policy for the metrics table. https://github.com/Verryx-02/RAM-USB/blob/17dffbd13729426591f7052f8e2b782d41332d67/services/metrics-collector/migrations/000001_create_metrics_table.up.sql#L56-L76, 2026.
- [62] Francesco Verrengia. RAM-USB: test che email inesistente e password errata producano lo stesso esito. https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/services/database-vault/internal/login/login_test.go#L88-L125, 2026.
- [63] golangci-lint. golangci-lint: analizzatore statico aggregato per Go. <https://golangci-lint.run/>, 2026.
- [64] securego. gosec: analizzatore di sicurezza statico per Go. <https://github.com/securego/gosec>, 2026.

- [65] timakin. bodyclose: rilevamento di corpi di risposta HTTP non chiusi. <https://github.com/timakin/bodyclose>, 2026.
- [66] ryanrolds. sqlclosecheck: rilevamento di righe e statement SQL non chiusi. <https://github.com/ryanrolds/sqlclosecheck>, 2026.
- [67] kkHAIKE. contextcheck: verifica della propagazione del context.Context. <https://github.com/kkHAIKE/contextcheck>, 2026.
- [68] polyfloyd. errorlint: verifica delle convenzioni di wrapping degli errori in Go. <https://github.com/polyfloyd/go-errorlint>, 2026.
- [69] Go Team. govulncheck: analisi delle vulnerabilità raggiungibili dal codice. <https://pkg.go.dev/golang.org/x/vuln/cmd/govulncheck>, 2026.
- [70] Gitleaks. gitleaks: rilevamento di segreti nei repository. <https://github.com/gitleaks/gitleaks>, 2026.
- [71] Aqua Security. trivy: scanner di vulnerabilità per immagini e filesystem. <https://github.com/aquasecurity/trivy>, 2026.
- [72] just-containers. s6-overlay: process supervisor for Docker containers, built on s6. <https://github.com/just-containers/s6-overlay>, 2026.
- [73] Francesco Verrengia. RAM-USB: Storage-service Proxmox deployment notes. <https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/deployments/proxmox/storage-service.md>, 2026.
- [74] Francesco Verrengia. RAM-USB: Storage-service's s6-overlay service directories. <https://github.com/Verryx-02/RAM-USB/tree/b7b2914230f844745df007057315c19f57d1a576/deployments/docker/storage-service/rootfs/etc/s6-overlay/s6-rc.d>, 2026.
- [75] Francesco Verrengia. RAM-USB: Entry-hub's distroless runtime stage. <https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/deployments/docker/entry-hub/Dockerfile#L100-L111>, 2026.
- [76] Francesco Verrengia. RAM-USB: Storage-service's debian:bookworm-slim runtime stage. <https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/deployments/docker/storage-service/Dockerfile#L60-L79>, 2026.
- [77] Proxmox Server Solutions GmbH. Proxmox VE: open-source server virtualization (KVM and LXC). <https://www.proxmox.com/en/proxmox-virtual-environment/overview>, 2026.
- [78] Francesco Verrengia. RAM-USB: Database-vault Proxmox deployment notes. <https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/deployments/proxmox/database-vault.md>, 2026.

- [79] Francesco Verrengia. RAM-USB: Security-switch Proxmox deployment notes. <https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/deployments/proxmox/security-switch.md>, 2026.
- [80] Francesco Verrengia. RAM-USB: one Docker Compose file per service. <https://github.com/Verryx-02/RAM-USB/tree/b7b2914230f844745df007057315c19f57d1a576/deployments/compose>, 2026.
- [81] Francesco Verrengia. RAM-USB: Entry-hub VPS deployment notes. <https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/deployments/vps/entry-hub.md>, 2026.
- [82] Francesco Verrengia. RAM-USB: HTTP-status-bound error constructors. <https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/pkg/errors/errors.go#L58-L104>, 2026.
- [83] Francesco Verrengia. RAM-USB: the Redacted log-value type. <https://github.com/Verryx-02/RAM-USB/blob/b7b2914230f844745df007057315c19f57d1a576/pkg/logging/logging.go#L24-L33>, 2026.
- [84] hashcat project. hashcat v7.0.0 release notes. https://github.com/hashcat/hashcat/blob/master/docs/releases_notes_v7.0.0.md, 2025.
- [85] Siwoo Eum, Hyunjun Kim, Minho Song, e Hwajeong Seo. Optimized implementation of Argon2 utilizing the graphics processing unit. *Applied Sciences*, 13(16):9295, 2023. <https://doi.org/10.3390/app13169295>.
- [86] Epoch AI. OpenAI Stargate Abilene: AI data center directory. <https://epoch.ai/data/ai-data-centers/directory/openai-stargate-abilene>, 2026. Dati al 18 agosto 2026.
- [87] NVIDIA. NVIDIA GB300 NVL72: specifiche tecniche. <https://www.nvidia.com/en-gb/data-center/gb300-nvl72>, 2026.
- [88] National Institute of Standards and Technology. Digital identity guidelines. <https://pages.nist.gov/800-63-4/sp800-63b.html>, 2025. NIST Special Publication 800-63B, revisione 4.